Comprehensive Notes on Matrices

For Machine Learning, Deep Learning & AI

Contents

1 Introduction to Matrices **3**
1.1 What are Matrices? . . . 3
1.2 Matrices in Machine Learning Context . . . 3

2 Types of Matrices **5**
2.1 Identity Matrix . . . 5
2.2 Zero Matrix . . . 6
2.3 Sparse Matrix . . . 7

3 Orthogonal Matrices **10**
3.1 Definition and Properties . . . 10
3.2 Example - 2x2 Orthogonal Matrix . . . 10
3.3 Example - 3x3 Orthogonal Matrix . . . 11
3.4 Use in Machine Learning . . . 11

4 Symmetric Matrices **15**
4.1 Definition and Properties . . . 15
4.2 Examples of Symmetric Matrices . . . 15
4.3 Use in Machine Learning . . . 16

5 Diagonal Matrices **20**
5.1 Definition and Properties . . . 20
5.2 Examples and Calculations . . . 20
5.3 Use in Machine Learning . . . 21

6 Matrix Equality **25**
6.1 Definition . . . 25
6.2 Examples . . . 25
6.3 Use in Machine Learning . . . 25

7 Summary **28**

8 Scalar Operations on Matrices **28**
8.1 Definition . . . 28
8.2 Properties of Scalar Operations . . . 29
8.3 Examples and Calculations . . . 29
8.4 Use in Machine Learning . . . 30

9 Matrix Addition and Subtraction **34**
9.1 Definition . . . 34
9.2 Properties . . . 34
9.3 Examples and Calculations . . . 34
9.4 Use in Machine Learning . . . 36

10 Matrix Multiplication **39**
10.1 Definition . . . 39
10.2 Properties . . . 39
10.3 Examples and Calculations . . . 39

10.4 Use in Machine Learning 41

11 Transpose of a Matrix **45**
11.1 Definition 45
11.2 Properties 45
11.3 Examples and Calculations 45

12 Determinant **50**
12.1 Definition 50
12.2 Formulas for Different Sizes 50
12.3 Properties 50
12.4 Examples and Calculations 51
12.5 Use in Machine Learning 52

13 Summary **55**

1 Introduction to Matrices

1.1 What are Matrices?

Definition: A matrix is a two-dimensional rectangular array of numbers, symbols, or expressions arranged in rows and columns. Matrices are fundamental mathematical structures used extensively in machine learning and deep learning.

Formal Notation: A matrix A of size $m \times n$ (m rows and n columns) is represented as:

$$A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix}_{m \times n} \tag{1}$$

where a_{ij} represents the element in the i-th row and j-th column.

1.2 Matrices in Machine Learning Context

In machine learning and deep learning, matrices serve multiple critical purposes:

1. Feature Representation:

Consider a dataset with n samples and d features. This is represented as a matrix X:

$$X = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1d} \\ x_{21} & x_{22} & \cdots & x_{2d} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{nd} \end{bmatrix}_{n \times d} \tag{2}$$

Example: Housing dataset with 5 samples and 3 features (area, bedrooms, age):

$$X = \begin{bmatrix} 1500 & 3 & 10 \\ 2000 & 4 & 5 \\ 1200 & 2 & 15 \\ 1800 & 3 & 8 \\ 2200 & 4 & 3 \end{bmatrix}_{5 \times 3} \tag{3}$$

Each row represents one house, and each column represents a feature.

2. Model Parameters:

In neural networks, weight matrices define transformations between layers. For a layer transforming input of dimension d_{in} to output of dimension d_{out}:

$$W = \begin{bmatrix} w_{11} & w_{12} & \cdots & w_{1d_{in}} \\ w_{21} & w_{22} & \cdots & w_{2d_{in}} \\ \vdots & \vdots & \ddots & \vdots \\ w_{d_{out}1} & w_{d_{out}2} & \cdots & w_{d_{out}d_{in}} \end{bmatrix}_{d_{out} \times d_{in}} \tag{4}$$

Example: Neural network layer with 4 input neurons and 3 output neurons:

$$W = \begin{bmatrix} 0.5 & -0.3 & 0.8 & 0.1 \\ -0.2 & 0.6 & -0.4 & 0.9 \\ 0.7 & 0.2 & -0.5 & -0.3 \end{bmatrix}_{3 \times 4} \tag{5}$$

3. Data Transformations:
Linear transformations in ML are represented as matrix operations:

$$Y = XW + b \tag{6}$$

where X is input data, W is weight matrix, b is bias vector, and Y is output.
Complete Calculation Example:
Given input $X = \begin{bmatrix} 2 & 3 \\ 1 & 4 \end{bmatrix}$ and weights $W = \begin{bmatrix} 0.5 & 0.2 \\ 0.3 & 0.6 \end{bmatrix}$:

$$\begin{aligned} Y &= XW && (7)\\ &= \begin{bmatrix} 2 & 3 \\ 1 & 4 \end{bmatrix} \begin{bmatrix} 0.5 & 0.2 \\ 0.3 & 0.6 \end{bmatrix} && (8)\\ &= \begin{bmatrix} 2(0.5) + 3(0.3) & 2(0.2) + 3(0.6) \\ 1(0.5) + 4(0.3) & 1(0.2) + 4(0.6) \end{bmatrix} && (9)\\ &= \begin{bmatrix} 1.0 + 0.9 & 0.4 + 1.8 \\ 0.5 + 1.2 & 0.2 + 2.4 \end{bmatrix} && (10)\\ &= \begin{bmatrix} 1.9 & 2.2 \\ 1.7 & 2.6 \end{bmatrix} && (11) \end{aligned}$$

2 Types of Matrices

2.1 Identity Matrix

Definition: An identity matrix I_n is a square matrix of size $n \times n$ where all diagonal elements are 1 and all off-diagonal elements are 0.

$$I_n = \begin{bmatrix} 1 & 0 & 0 & \cdots & 0 \\ 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & 1 \end{bmatrix}_{n \times n} \tag{12}$$

Mathematical Property: For any matrix A of size $m \times n$:

$$A \cdot I_n = I_m \cdot A = A \tag{13}$$

Example - 3x3 Identity Matrix:

$$I_3 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \tag{14}$$

Verification Calculation:

Let $A = \begin{bmatrix} 2 & 3 & 1 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$

$$A \cdot I_3 = \begin{bmatrix} 2 & 3 & 1 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \tag{15}$$

$$= \begin{bmatrix} 2(1)+3(0)+1(0) & 2(0)+3(1)+1(0) & 2(0)+3(0)+1(1) \\ 4(1)+5(0)+6(0) & 4(0)+5(1)+6(0) & 4(0)+5(0)+6(1) \\ 7(1)+8(0)+9(0) & 7(0)+8(1)+9(0) & 7(0)+8(0)+9(1) \end{bmatrix} \tag{16}$$

$$= \begin{bmatrix} 2 & 3 & 1 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} = A \tag{17}$$

Use in Machine Learning:

1. Linear Algebra Operations: Identity matrices are used in solving linear systems and computing matrix inverses.

2. Regularization: In Ridge regression, the cost function includes an identity matrix term:

$$\theta = (X^T X + \lambda I)^{-1} X^T y \tag{18}$$

Example - Ridge Regression Calculation:

Given $X = \begin{bmatrix} 1 & 2 \\ 1 & 3 \\ 1 & 4 \end{bmatrix}$, $y = \begin{bmatrix} 3 \\ 5 \\ 7 \end{bmatrix}$, and $\lambda = 0.1$:

Step 1: Calculate $X^T X$:

$$X^T X = \begin{bmatrix} 1 & 1 & 1 \\ 2 & 3 & 4 \end{bmatrix} \begin{bmatrix} 1 & 2 \\ 1 & 3 \\ 1 & 4 \end{bmatrix} \tag{19}$$

$$= \begin{bmatrix} 3 & 9 \\ 9 & 29 \end{bmatrix} \tag{20}$$

Step 2: Add regularization term λI:

$$X^T X + \lambda I = \begin{bmatrix} 3 & 9 \\ 9 & 29 \end{bmatrix} + 0.1 \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 3.1 & 9 \\ 9 & 29.1 \end{bmatrix} \tag{21}$$

This regularization prevents overfitting by adding a penalty term to the weights.

3. Initialization in Neural Networks: Identity matrices are sometimes used to initialize recurrent neural network (RNN) weights to help with gradient flow.

2.2 Zero Matrix

Definition: A zero matrix (or null matrix) is a matrix where all elements are zero.

$$0_{m \times n} = \begin{bmatrix} 0 & 0 & \cdots & 0 \\ 0 & 0 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 0 \end{bmatrix}_{m \times n} \tag{22}$$

Mathematical Properties:

$$A + 0 = A \tag{23}$$

$$A \cdot 0 = 0 \tag{24}$$

$$0 \cdot A = 0 \tag{25}$$

Example - 2x3 Zero Matrix:

$$0_{2 \times 3} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \tag{26}$$

Use in Machine Learning:

1. Bias Initialization: Bias vectors in neural networks are often initialized as zero vectors (which can be viewed as $n \times 1$ zero matrices).

Example: For a neural network layer with 4 neurons:

$$b = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} \tag{27}$$

2. Padding in CNNs: Zero matrices are used for padding in convolutional neural networks.

Example - Zero Padding:

Original image: $\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$

After padding with zeros (padding=1):

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 2 & 3 & 0 \\ 0 & 4 & 5 & 6 & 0 \\ 0 & 7 & 8 & 9 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \tag{28}$$

3. Masking in NLP: Zero matrices are used to mask padded tokens in sequence models.

Example: Attention mask for sequence ["Hello", "World", PAD, PAD]:

$$\text{Mask} = \begin{bmatrix} 1 & 1 & 0 & 0 \end{bmatrix} \tag{29}$$

where 1 indicates real tokens and 0 indicates padding.

2.3 Sparse Matrix

Definition: A sparse matrix is a matrix in which most elements are zero. Formally, a matrix is considered sparse if the number of non-zero elements is significantly smaller than the total number of elements.

Sparsity Ratio:

$$\text{Sparsity} = \frac{\text{Number of zero elements}}{\text{Total number of elements}} = \frac{|zeros|}{m \times n} \tag{30}$$

Example - Sparse Matrix:

$$S = \begin{bmatrix} 0 & 0 & 3 & 0 & 0 \\ 0 & 0 & 0 & 0 & 7 \\ 2 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 5 & 0 \\ 0 & 4 & 0 & 0 & 0 \end{bmatrix}_{5\times5} \tag{31}$$

Sparsity calculation:

$$\text{Total elements} = 5 \times 5 = 25 \tag{32}$$

$$\text{Non-zero elements} = 5 \tag{33}$$

$$\text{Zero elements} = 20 \tag{34}$$

$$\text{Sparsity} = \frac{20}{25} = 0.8 = 80\% \tag{35}$$

Sparse Matrix Storage Formats:

1. COO (Coordinate) Format:

Stores (row, column, value) triplets for each non-zero element:

For the matrix S above:

$$\text{row} = [0, 1, 2, 3, 4] \tag{36}$$

$$\text{col} = [2, 4, 0, 3, 1] \tag{37}$$

$$\text{data} = [3, 7, 2, 5, 4] \tag{38}$$

Memory Savings:

$$\text{Dense storage} = 25 \times 4 \text{ bytes} = 100 \text{ bytes (float32)} \tag{39}$$

$$\text{COO storage} = 5 \times 3 \times 4 \text{ bytes} = 60 \text{ bytes} \tag{40}$$

2. CSR (Compressed Sparse Row) Format:
More efficient for row-wise operations:

$$\text{data} = [3, 7, 2, 5, 4] \tag{41}$$

$$\text{indices} = [2, 4, 0, 3, 1] \tag{42}$$

$$\text{indptr} = [0, 1, 2, 3, 4, 5] \tag{43}$$

Use in Machine Learning:
1. Text Data Representation: TF-IDF matrices and bag-of-words representations are typically sparse.
Example - Document-Term Matrix:
Consider 4 documents and vocabulary of 6 words:

$$\text{DTM} = \begin{bmatrix} 2 & 0 & 1 & 0 & 0 & 0 \\ 0 & 3 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 2 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 4 \end{bmatrix}_{4\times 6} \tag{44}$$

Sparsity: $\frac{18}{24} = 75\%$ sparse
In real-world NLP, vocabulary sizes are 10,000-100,000 words, making sparsity often $> 99\%$.
2. Recommendation Systems: User-item interaction matrices are extremely sparse.
Example - User-Movie Rating Matrix:

$$R = \begin{bmatrix} 5 & 0 & 0 & 3 & 0 & 0 \\ 0 & 4 & 0 & 0 & 0 & 5 \\ 0 & 0 & 0 & 0 & 3 & 0 \\ 4 & 0 & 2 & 0 & 0 & 0 \\ 0 & 0 & 0 & 5 & 0 & 4 \end{bmatrix}_{5\times 6} \tag{45}$$

Users: 5, Movies: 6, Ratings: 10, Sparsity: $\frac{20}{30} = 66.7\%$
In real systems (Netflix: 500K users, 20K movies), sparsity is typically $> 99.5\%$.
3. Neural Network Pruning: Sparsity is induced in neural networks to reduce computational cost.
Example - Weight Pruning:
Original weight matrix:

$$W = \begin{bmatrix} 0.23 & -0.45 & 0.67 \\ 0.12 & 0.89 & -0.34 \\ -0.56 & 0.01 & 0.78 \end{bmatrix} \tag{46}$$

After pruning (threshold = 0.3):

$$W_{\text{pruned}} = \begin{bmatrix} 0 & -0.45 & 0.67 \\ 0 & 0.89 & -0.34 \\ -0.56 & 0 & 0.78 \end{bmatrix} \tag{47}$$

Sparsity: $\frac{3}{9} = 33.3\%$

Computational Benefits:

For matrix-vector multiplication $y = Wx$ where W is $m \times n$ sparse:

Dense: $O(mn)$ operations

Sparse (with k non-zeros): $O(k)$ operations

Example Calculation:

Sparse matrix $S = \begin{bmatrix} 0 & 3 & 0 \\ 2 & 0 & 0 \\ 0 & 0 & 5 \end{bmatrix}$, vector $x = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$

$$y = Sx \tag{48}$$

$$y_1 = 0(1) + 3(2) + 0(3) = 6 \tag{49}$$

$$y_2 = 2(1) + 0(2) + 0(3) = 2 \tag{50}$$

$$y_3 = 0(1) + 0(2) + 5(3) = 15 \tag{51}$$

$$y = \begin{bmatrix} 6 \\ 2 \\ 15 \end{bmatrix} \tag{52}$$

Operations: Only 3 multiplications instead of 9 (66.7% reduction).

3 Orthogonal Matrices

3.1 Definition and Properties

Definition: A square matrix Q is orthogonal if its transpose equals its inverse:

$$Q^T Q = QQ^T = I \tag{53}$$

Equivalently:

$$Q^T = Q^{-1} \tag{54}$$

Column/Row Property: The columns (and rows) of an orthogonal matrix form an orthonormal set of vectors.

For columns $q_1, q_2, \ldots, q_n$:

$$q_i^T q_j = \begin{cases} 1 & \text{if } i = j \\ 0 & \text{if } i \neq j \end{cases} = \delta_{ij} \tag{55}$$

where δ_{ij} is the Kronecker delta.

Geometric Property: Orthogonal matrices preserve lengths and angles:

$$||Qx||_2 = ||x||_2 \quad \text{(length preservation)} \tag{56}$$

$$(Qx)^T(Qy) = x^T y \quad \text{(angle preservation)} \tag{57}$$

3.2 Example - 2x2 Orthogonal Matrix

Rotation Matrix:

$$Q = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \tag{58}$$

For $\theta = 45$:

$$Q = \begin{bmatrix} \frac{\sqrt{2}}{2} & -\frac{\sqrt{2}}{2} \\ \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} \end{bmatrix} \tag{59}$$

Verification that Q is Orthogonal:

Step 1: Calculate Q^T:

$$Q^T = \begin{bmatrix} \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} \\ -\frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} \end{bmatrix} \tag{60}$$

Step 2: Calculate $Q^T Q$:

$$Q^T Q = \begin{bmatrix} \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} \\ -\frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} \end{bmatrix} \begin{bmatrix} \frac{\sqrt{2}}{2} & -\frac{\sqrt{2}}{2} \\ \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} \end{bmatrix} \tag{61}$$

$$= \begin{bmatrix} \frac{1}{2} + \frac{1}{2} & -\frac{1}{2} + \frac{1}{2} \\ -\frac{1}{2} + \frac{1}{2} & \frac{1}{2} + \frac{1}{2} \end{bmatrix} \tag{62}$$

$$= \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = I \tag{63}$$

Verification of Column Orthonormality:

Column 1: $q_1 = \begin{bmatrix} \frac{\sqrt{2}}{2} \\ \frac{\sqrt{2}}{2} \end{bmatrix}$, Column 2: $q_2 = \begin{bmatrix} -\frac{\sqrt{2}}{2} \\ \frac{\sqrt{2}}{2} \end{bmatrix}$

$$||q_1||_2 = \sqrt{\left(\frac{\sqrt{2}}{2}\right)^2 + \left(\frac{\sqrt{2}}{2}\right)^2} = \sqrt{\frac{1}{2} + \frac{1}{2}} = 1 \tag{64}$$

$$||q_2||_2 = \sqrt{\left(-\frac{\sqrt{2}}{2}\right)^2 + \left(\frac{\sqrt{2}}{2}\right)^2} = \sqrt{\frac{1}{2} + \frac{1}{2}} = 1 \tag{65}$$

$$q_1^T q_2 = \frac{\sqrt{2}}{2} \cdot \left(-\frac{\sqrt{2}}{2}\right) + \frac{\sqrt{2}}{2} \cdot \frac{\sqrt{2}}{2} = -\frac{1}{2} + \frac{1}{2} = 0 \tag{66}$$

Columns are orthonormal

3.3 Example - 3x3 Orthogonal Matrix

$$Q = \begin{bmatrix} \frac{1}{\sqrt{3}} & \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{6}} \\ \frac{1}{\sqrt{3}} & 0 & -\frac{2}{\sqrt{6}} \\ \frac{1}{\sqrt{3}} & -\frac{1}{\sqrt{2}} & \frac{1}{\sqrt{6}} \end{bmatrix} \tag{67}$$

Full Verification:
Compute $Q^T Q$:

$$(Q^T Q)_{11} = \frac{1}{3} + \frac{1}{3} + \frac{1}{3} = 1 \tag{68}$$

$$(Q^T Q)_{12} = \frac{1}{\sqrt{6}} + 0 - \frac{1}{\sqrt{6}} = 0 \tag{69}$$

$$(Q^T Q)_{13} = \frac{1}{3\sqrt{2}} - \frac{2}{3\sqrt{2}} + \frac{1}{3\sqrt{2}} = 0 \tag{70}$$

$$(Q^T Q)_{22} = \frac{1}{2} + 0 + \frac{1}{2} = 1 \tag{71}$$

$$(Q^T Q)_{23} = \frac{1}{2\sqrt{3}} + 0 - \frac{1}{2\sqrt{3}} = 0 \tag{72}$$

$$(Q^T Q)_{33} = \frac{1}{6} + \frac{4}{6} + \frac{1}{6} = 1 \tag{73}$$

Therefore $Q^T Q = I$

3.4 Use in Machine Learning

1. Principal Component Analysis (PCA):
PCA involves eigendecomposition of the covariance matrix:

$$\Sigma = Q \Lambda Q^T \tag{74}$$

where Q is an orthogonal matrix of eigenvectors, and Λ is a diagonal matrix of eigenvalues.

Complete PCA Example:

Dataset: $X = \begin{bmatrix} 2 & 1 \\ 3 & 3 \\ 5 & 4 \end{bmatrix}$ (3 samples, 2 features)

Step 1: Center the data:

$$\mu = \begin{bmatrix} \frac{2+3+5}{3} \\ \frac{1+3+4}{3} \end{bmatrix} = \begin{bmatrix} 3.33 \\ 2.67 \end{bmatrix} \tag{75}$$

$$X_{\text{centered}} = X - \mu = \begin{bmatrix} -1.33 & -1.67 \\ -0.33 & 0.33 \\ 1.67 & 1.33 \end{bmatrix} \tag{76}$$

Step 2: Calculate covariance matrix:

$$\Sigma = \frac{1}{n-1} X_{\text{centered}}^T X_{\text{centered}} \tag{77}$$

$$= \frac{1}{2} \begin{bmatrix} -1.33 & -0.33 & 1.67 \\ -1.67 & 0.33 & 1.33 \end{bmatrix} \begin{bmatrix} -1.33 & -1.67 \\ -0.33 & 0.33 \\ 1.67 & 1.33 \end{bmatrix} \tag{78}$$

$$= \frac{1}{2} \begin{bmatrix} 4.67 & 4.44 \\ 4.44 & 4.56 \end{bmatrix} = \begin{bmatrix} 2.33 & 2.22 \\ 2.22 & 2.28 \end{bmatrix} \tag{79}$$

Step 3: Find eigenvalues and eigenvectors:
Solving $\det(\Sigma - \lambda I) = 0$:

$$\lambda_1 = 4.55, \quad \lambda_2 = 0.06 \tag{80}$$

$$v_1 = \begin{bmatrix} 0.708 \\ 0.706 \end{bmatrix}, \quad v_2 = \begin{bmatrix} -0.706 \\ 0.708 \end{bmatrix} \tag{81}$$

Step 4: Form orthogonal matrix Q:

$$Q = \begin{bmatrix} 0.708 & -0.706 \\ 0.706 & 0.708 \end{bmatrix} \tag{82}$$

Verification:

$$Q^T Q = \begin{bmatrix} 0.708 & 0.706 \\ -0.706 & 0.708 \end{bmatrix} \begin{bmatrix} 0.708 & -0.706 \\ 0.706 & 0.708 \end{bmatrix} \tag{83}$$

$$= \begin{bmatrix} 1.0 & 0.0 \\ 0.0 & 1.0 \end{bmatrix} = I \tag{84}$$

Step 5: Project data onto principal components:

$$X_{\text{PCA}} = X_{\text{centered}} Q \tag{85}$$

$$= \begin{bmatrix} -1.33 & -1.67 \\ -0.33 & 0.33 \\ 1.67 & 1.33 \end{bmatrix} \begin{bmatrix} 0.708 & -0.706 \\ 0.706 & 0.708 \end{bmatrix} \tag{86}$$

$$= \begin{bmatrix} -2.12 & 0.24 \\ 0.00 & -0.47 \\ 2.12 & 0.23 \end{bmatrix} \tag{87}$$

The first column contains projections onto the principal component (direction of maximum variance).

2. Singular Value Decomposition (SVD):

Any matrix A can be decomposed as:

$$A = U\Sigma V^T \tag{88}$$

where U and V are orthogonal matrices, and Σ is diagonal.

SVD Example:

Given $A = \begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix}$

Solution:

$$U = \begin{bmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} \end{bmatrix}, \quad \Sigma = \begin{bmatrix} 4 & 0 \\ 0 & 2 \end{bmatrix} \tag{89}$$

$$V = \begin{bmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} \end{bmatrix} \tag{90}$$

Verification:

$$U\Sigma V^T = \begin{bmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} \end{bmatrix} \begin{bmatrix} 4 & 0 \\ 0 & 2 \end{bmatrix} \begin{bmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} \end{bmatrix} \tag{91}$$

$$= \begin{bmatrix} \frac{4}{\sqrt{2}} & \frac{2}{\sqrt{2}} \\ \frac{4}{\sqrt{2}} & -\frac{2}{\sqrt{2}} \end{bmatrix} \begin{bmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} \end{bmatrix} \tag{92}$$

$$= \begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix} = A \tag{93}$$

Applications in ML:

- **Dimensionality Reduction:** Truncated SVD for reducing feature space
- **Latent Semantic Analysis:** In NLP for topic modeling
- **Recommendation Systems:** Matrix factorization for collaborative filtering
- **Image Compression:** Approximating images with fewer singular values

3. Orthogonal Weight Initialization in Deep Learning:

Initializing weights with orthogonal matrices helps maintain gradient flow in deep networks.

Problem: Random initialization can lead to vanishing/exploding gradients.

Solution: Initialize weight matrix W as orthogonal:

$$W^T W = I \implies ||Wx||_2 = ||x||_2 \tag{94}$$

This preserves the norm of activations through layers.

Example - Recurrent Neural Network (RNN):

RNN hidden state update:

$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b) \tag{95}$$

If W_h is orthogonal, gradients flow better through time steps.

Practical Example:

Initialize $W_h \in \mathbb{R}^{3\times3}$ orthogonally:

$$W_h = \begin{bmatrix} 0.577 & 0.577 & 0.577 \\ 0.707 & -0.707 & 0.000 \\ 0.408 & 0.408 & -0.816 \end{bmatrix} \tag{96}$$

Verification of orthogonality and gradient flow properties show that $||W_h h||_2 = ||h||_2$ for any h.

4. QR Decomposition:

Any matrix A can be decomposed as:

$$A = QR \tag{97}$$

where Q is orthogonal and R is upper triangular.

Example:

Given $A = \begin{bmatrix} 1 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix}$

Using Gram-Schmidt process:

Step 1: $q_1 = \frac{a_1}{||a_1||} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix}$

Step 2: $u_2 = a_2 - (a_2^T q_1)q_1 = \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} - \frac{1}{2} \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0.5 \\ -0.5 \\ 1 \end{bmatrix}$

$q_2 = \frac{u_2}{||u_2||} = \frac{1}{\sqrt{1.5}} \begin{bmatrix} 0.5 \\ -0.5 \\ 1 \end{bmatrix}$

Continue for q_3 to get complete Q matrix.

Use in ML: Solving least squares problems in linear regression efficiently.

4 Symmetric Matrices

4.1 Definition and Properties

Definition: A square matrix A is symmetric if it equals its transpose:

$$A = A^T \tag{98}$$

Equivalently, $a_{ij} = a_{ji}$ for all i, j.

Key Properties:

1. Real Eigenvalues: All eigenvalues of a real symmetric matrix are real numbers.

2. Orthogonal Eigenvectors: Eigenvectors corresponding to distinct eigenvalues are orthogonal.

3. Spectral Theorem: A symmetric matrix can be diagonalized by an orthogonal matrix:

$$A = Q\Lambda Q^T \tag{99}$$

where Q is orthogonal ($Q^TQ = I$) and Λ is diagonal.

4.2 Examples of Symmetric Matrices

Example 1 - 2x2 Symmetric Matrix:

$$A = \begin{bmatrix} 4 & 2 \\ 2 & 3 \end{bmatrix} \tag{100}$$

Verification: $A^T = \begin{bmatrix} 4 & 2 \\ 2 & 3 \end{bmatrix} = A$

Finding Eigenvalues:

Characteristic equation: $\det(A - \lambda I) = 0$

$$\det \begin{bmatrix} 4-\lambda & 2 \\ 2 & 3-\lambda \end{bmatrix} = 0 \tag{101}$$

$$(4-\lambda)(3-\lambda) - 4 = 0 \tag{102}$$

$$\lambda^2 - 7\lambda + 8 = 0 \tag{103}$$

$$\lambda = \frac{7 \pm \sqrt{49 - 32}}{2} = \frac{7 \pm \sqrt{17}}{2} \tag{104}$$

$$\lambda_1 = \frac{7 + \sqrt{17}}{2} \approx 5.56 \tag{105}$$

$$\lambda_2 = \frac{7 - \sqrt{17}}{2} \approx 1.44 \tag{106}$$

Both eigenvalues are real

Finding Eigenvectors:

For $\lambda_1 = 5.56$:

$$(A - 5.56I)v_1 = 0 \tag{107}$$

$$\begin{bmatrix} -1.56 & 2 \\ 2 & -2.56 \end{bmatrix} v_1 = 0 \tag{108}$$

Solution: $v_1 = \begin{bmatrix} 0.78 \\ 0.62 \end{bmatrix}$ (normalized)

For $\lambda_2 = 1.44$:

$$v_2 = \begin{bmatrix} -0.62 \\ 0.78 \end{bmatrix} \tag{109}$$

Verification of Orthogonality:

$$v_1^T v_2 = 0.78(-0.62) + 0.62(0.78) = -0.48 + 0.48 = 0 \tag{110}$$

Example 2 - 3x3 Symmetric Matrix:

$$A = \begin{bmatrix} 2 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 2 \end{bmatrix} \tag{111}$$

This is a tridiagonal symmetric matrix (common in numerical analysis).

Eigenvalue calculation:

$$\lambda_1 = 2 + \sqrt{2} \approx 3.41 \tag{112}$$

$$\lambda_2 = 2 \tag{113}$$

$$\lambda_3 = 2 - \sqrt{2} \approx 0.59 \tag{114}$$

All real eigenvalues

4.3 Use in Machine Learning

1. Covariance Matrices:

The covariance matrix of a dataset is always symmetric:

$$\Sigma = \frac{1}{n-1} X^T X \tag{115}$$

where X is the centered data matrix.

Complete Example - Covariance Matrix Calculation:

Dataset: $X = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 7 \\ 7 & 8 \end{bmatrix}$ (4 samples, 2 features)

Step 1: Calculate means:

$$\mu = \begin{bmatrix} \frac{1+3+5+7}{4} \\ \frac{2+4+7+8}{4} \end{bmatrix} = \begin{bmatrix} 4 \\ 5.25 \end{bmatrix} \tag{116}$$

Step 2: Center the data:

$$X_{\text{centered}} = \begin{bmatrix} -3 & -3.25 \\ -1 & -1.25 \\ 1 & 1.75 \\ 3 & 2.75 \end{bmatrix} \tag{117}$$

Step 3: Calculate covariance matrix:

$$\Sigma = \frac{1}{4-1} X_{\text{centered}}^T X_{\text{centered}} \tag{118}$$

$$= \frac{1}{3} \begin{bmatrix} -3 & -1 & 1 & 3 \\ -3.25 & -1.25 & 1.75 & 2.75 \end{bmatrix} \begin{bmatrix} -3 & -3.25 \\ -1 & -1.25 \\ 1 & 1.75 \\ 3 & 2.75 \end{bmatrix} \tag{119}$$

$$= \frac{1}{3} \begin{bmatrix} 20 & 22 \\ 22 & 24.75 \end{bmatrix} \tag{120}$$

$$= \begin{bmatrix} 6.67 & 7.33 \\ 7.33 & 8.25 \end{bmatrix} \tag{121}$$

Verification of Symmetry: $\Sigma^T = \Sigma$
Interpretation:

- Diagonal elements (6.67, 8.25) are variances of features 1 and 2
- Off-diagonal element (7.33) is covariance between features
- Positive covariance indicates features tend to increase together

Eigendecomposition for PCA:
Eigenvalues: $\lambda_1 = 14.64$, $\lambda_2 = 0.28$
Explained variance ratio: $\frac{14.64}{14.64+0.28} = 98.1\%$ captured by first PC.
2. Kernel Matrices in SVM:
The Gram matrix K in kernel methods is symmetric:

$$K_{ij} = k(x_i, x_j) = k(x_j, x_i) = K_{ji} \tag{122}$$

Example - RBF Kernel Matrix:
Given 3 data points: $x_1 = [1, 2]$, $x_2 = [2, 3]$, $x_3 = [4, 5]$
RBF kernel: $k(x_i, x_j) = \exp(-\gamma ||x_i - x_j||^2)$ with $\gamma = 0.5$
Calculations:

$$K_{11} = k(x_1, x_1) = \exp(0) = 1.0 \tag{123}$$

$$K_{12} = k(x_1, x_2) = \exp(-0.5 \cdot ((1-2)^2 + (2-3)^2)) = \exp(-1.0) \approx 0.368 \tag{124}$$

$$K_{13} = k(x_1, x_3) = \exp(-0.5 \cdot ((1-4)^2 + (2-5)^2)) = \exp(-9.0) \approx 0.0001 \tag{125}$$

$$K_{22} = 1.0 \tag{126}$$

$$K_{23} = \exp(-0.5 \cdot ((2-4)^2 + (3-5)^2)) = \exp(-4.0) \approx 0.018 \tag{127}$$

$$K_{33} = 1.0 \tag{128}$$

Kernel Matrix:

$$K = \begin{bmatrix} 1.000 & 0.368 & 0.0001 \\ 0.368 & 1.000 & 0.018 \\ 0.0001 & 0.018 & 1.000 \end{bmatrix} \tag{129}$$

Verification: $K^T = K$ (symmetric)
Properties:

- Diagonal elements are always 1 (self-similarity)
- Off-diagonal elements measure similarity between different points
- Symmetric because kernel function is symmetric
- Positive semi-definite (all eigenvalues 0)

3. Hessian Matrix in Optimization:
The Hessian matrix of second derivatives is symmetric:

$$H_{ij} = \frac{\partial^2 f}{\partial x_i \partial x_j} = \frac{\partial^2 f}{\partial x_j \partial x_i} = H_{ji} \tag{130}$$

Example - Quadratic Function:
Consider $f(x_1, x_2) = x_1^2 + 2x_1x_2 + 3x_2^2$
First Derivatives:

$$\frac{\partial f}{\partial x_1} = 2x_1 + 2x_2 \tag{131}$$
$$\frac{\partial f}{\partial x_2} = 2x_1 + 6x_2 \tag{132}$$

Second Derivatives (Hessian):

$$\frac{\partial^2 f}{\partial x_1^2} = 2 \tag{133}$$
$$\frac{\partial^2 f}{\partial x_1 \partial x_2} = 2 \tag{134}$$
$$\frac{\partial^2 f}{\partial x_2 \partial x_1} = 2 \tag{135}$$
$$\frac{\partial^2 f}{\partial x_2^2} = 6 \tag{136}$$

Hessian Matrix:

$$H = \begin{bmatrix} 2 & 2 \\ 2 & 6 \end{bmatrix} \tag{137}$$

Eigenvalues:

$$\det(H - \lambda I) = (2 - \lambda)(6 - \lambda) - 4 = 0 \tag{138}$$
$$\lambda^2 - 8\lambda + 8 = 0 \tag{139}$$
$$\lambda_1 = 4 + 2\sqrt{2} \approx 6.83 \tag{140}$$
$$\lambda_2 = 4 - 2\sqrt{2} \approx 1.17 \tag{141}$$

Both eigenvalues positive $\rightarrow$ H is positive definite $\rightarrow$ function has a minimum.
Use in Gradient Descent:
Newton's method uses the Hessian:

$$x_{k+1} = x_k - H^{-1}\nabla f(x_k) \tag{142}$$

At point $x = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$:

$$\nabla f = \begin{bmatrix} 2(1) + 2(1) \\ 2(1) + 6(1) \end{bmatrix} = \begin{bmatrix} 4 \\ 8 \end{bmatrix} \tag{143}$$

$$H^{-1} = \frac{1}{8} \begin{bmatrix} 6 & -2 \\ -2 & 2 \end{bmatrix} = \begin{bmatrix} 0.75 & -0.25 \\ -0.25 & 0.25 \end{bmatrix} \tag{144}$$

$$x_{\text{new}} = \begin{bmatrix} 1 \\ 1 \end{bmatrix} - \begin{bmatrix} 0.75 & -0.25 \\ -0.25 & 0.25 \end{bmatrix} \begin{bmatrix} 4 \\ 8 \end{bmatrix} \tag{145}$$

$$= \begin{bmatrix} 1 \\ 1 \end{bmatrix} - \begin{bmatrix} 1.0 \\ 1.0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \tag{146}$$

Converges to minimum in one step (exact for quadratic functions).

4. Graph Laplacian in Spectral Clustering:

The graph Laplacian matrix $L = D - A$ is symmetric, where:

- D is the degree matrix (diagonal)
- A is the adjacency matrix (symmetric)

Example:

Graph with 4 nodes and edges: (1,2), (2,3), (3,4), (1,4)

Adjacency Matrix:

$$A = \begin{bmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{bmatrix} \tag{147}$$

Degree Matrix:

$$D = \begin{bmatrix} 2 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 \\ 0 & 0 & 2 & 0 \\ 0 & 0 & 0 & 2 \end{bmatrix} \tag{148}$$

Laplacian Matrix:

$$L = D - A = \begin{bmatrix} 2 & -1 & 0 & -1 \\ -1 & 2 & -1 & 0 \\ 0 & -1 & 2 & -1 \\ -1 & 0 & -1 & 2 \end{bmatrix} \tag{149}$$

Properties:

- Symmetric: $L^T = L$
- Row sums are zero: $L\mathbf{1} = \mathbf{0}$
- Positive semi-definite: all eigenvalues ≥ 0
- Number of zero eigenvalues = number of connected components

This matrix is used in spectral clustering to find community structure in graphs.

5 Diagonal Matrices

5.1 Definition and Properties

Definition: A diagonal matrix is a square matrix where all off-diagonal elements are zero:

$$D = \begin{bmatrix} d_{11} & 0 & 0 & \cdots & 0 \\ 0 & d_{22} & 0 & \cdots & 0 \\ 0 & 0 & d_{33} & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & d_{nn} \end{bmatrix} \tag{150}$$

Compact notation: $D = \text{diag}(d_{11}, d_{22}, \ldots, d_{nn})$

Key Properties:

1. **Matrix Multiplication:** Multiplying by a diagonal matrix scales rows or columns:

$$D \cdot A \text{ scales rows of } A \tag{151}$$

$$A \cdot D \text{ scales columns of } A \tag{152}$$

2. **Inverse:** Easy to compute (if all diagonal elements are non-zero):

$$D^{-1} = \text{diag}\left(\frac{1}{d_{11}}, \frac{1}{d_{22}}, \ldots, \frac{1}{d_{nn}}\right) \tag{153}$$

3. **Powers:**

$$D^k = \text{diag}(d_{11}^k, d_{22}^k, \ldots, d_{nn}^k) \tag{154}$$

4. **Eigenvalues:** Diagonal elements are the eigenvalues.

5.2 Examples and Calculations

Example 1 - Basic Diagonal Matrix:

$$D = \begin{bmatrix} 3 & 0 & 0 \\ 0 & 5 & 0 \\ 0 & 0 & 2 \end{bmatrix} \tag{155}$$

Inverse:

$$D^{-1} = \begin{bmatrix} \frac{1}{3} & 0 & 0 \\ 0 & \frac{1}{5} & 0 \\ 0 & 0 & \frac{1}{2} \end{bmatrix} = \begin{bmatrix} 0.333 & 0 & 0 \\ 0 & 0.2 & 0 \\ 0 & 0 & 0.5 \end{bmatrix} \tag{156}$$

Verification:

$$D \cdot D^{-1} = \begin{bmatrix} 3 & 0 & 0 \\ 0 & 5 & 0 \\ 0 & 0 & 2 \end{bmatrix} \begin{bmatrix} 0.333 & 0 & 0 \\ 0 & 0.2 & 0 \\ 0 & 0 & 0.5 \end{bmatrix} \tag{157}$$

$$= \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = I \tag{158}$$

Square:

$$D^2 = \begin{bmatrix} 9 & 0 & 0 \\ 0 & 25 & 0 \\ 0 & 0 & 4 \end{bmatrix} \tag{159}$$

Example 2 - Scaling Operations:
Row Scaling:
$D = \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix}$, $A = \begin{bmatrix} 1 & 4 \\ 2 & 5 \end{bmatrix}$

$$DA = \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix} \begin{bmatrix} 1 & 4 \\ 2 & 5 \end{bmatrix} \tag{160}$$

$$= \begin{bmatrix} 2(1) + 0(2) & 2(4) + 0(5) \\ 0(1) + 3(2) & 0(4) + 3(5) \end{bmatrix} \tag{161}$$

$$= \begin{bmatrix} 2 & 8 \\ 6 & 15 \end{bmatrix} \tag{162}$$

Row 1 scaled by 2, Row 2 scaled by 3
Column Scaling:

$$AD = \begin{bmatrix} 1 & 4 \\ 2 & 5 \end{bmatrix} \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix} \tag{163}$$

$$= \begin{bmatrix} 1(2) + 4(0) & 1(0) + 4(3) \\ 2(2) + 5(0) & 2(0) + 5(3) \end{bmatrix} \tag{164}$$

$$= \begin{bmatrix} 2 & 12 \\ 4 & 15 \end{bmatrix} \tag{165}$$

Column 1 scaled by 2, Column 2 scaled by 3

5.3 Use in Machine Learning

1. Feature Scaling:
Diagonal matrices are used for scaling features to have similar ranges.
Example - Min-Max Scaling:
Dataset: $X = \begin{bmatrix} 10 & 200 \\ 20 & 400 \\ 30 & 600 \end{bmatrix}$
Feature ranges: [10, 30] and [200, 600]
Goal: Scale to [0, 1]
Scaling factors:

$$s_1 = \frac{1}{30 - 10} = 0.05 \tag{166}$$

$$s_2 = \frac{1}{600 - 200} = 0.0025 \tag{167}$$

Scaling matrix:

$$S = \begin{bmatrix} 0.05 & 0 \\ 0 & 0.0025 \end{bmatrix} \tag{168}$$

First shift minimum to zero:

$$X_{\text{shifted}} = X - \begin{bmatrix}10 & 200\end{bmatrix} = \begin{bmatrix}0 & 0\\ 10 & 200\\ 20 & 400\end{bmatrix} \tag{169}$$

Then scale:

$$X_{\text{scaled}} = X_{\text{shifted}} S \tag{170}$$

$$= \begin{bmatrix}0 & 0\\ 10 & 200\\ 20 & 400\end{bmatrix} \begin{bmatrix}0.05 & 0\\ 0 & 0.0025\end{bmatrix} \tag{171}$$

$$= \begin{bmatrix}0 & 0\\ 0.5 & 0.5\\ 1.0 & 1.0\end{bmatrix} \tag{172}$$

All features now in range [0, 1]

2. Learning Rate Schedules in Optimization:

Adaptive learning rate methods use diagonal matrices to scale gradients differently foreach parameter.

Example - Diagonal Preconditioning in AdaGrad:

AdaGrad accumulates squared gradients:

$$G_t = G_{t-1} + g_t \odot g_t \tag{173}$$

where $\odot$ is element-wise product, and g_t is gradient at step t.

Update rule:

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{G_t + \epsilon}} \odot g_t \tag{174}$$

Complete Calculation Example:

Parameters: $\theta_0 = \begin{bmatrix}1.0\\ 2.0\end{bmatrix}$, $\eta = 0.1$, $\epsilon = 10^{-8}$

Gradients over 3 steps:

$$g_1 = \begin{bmatrix}0.5\\ 0.3\end{bmatrix}, \quad g_2 = \begin{bmatrix}0.4\\ 0.6\end{bmatrix}, \quad g_3 = \begin{bmatrix}0.3\\ 0.2\end{bmatrix} \tag{175}$$

Step 1:

$$G_1 = \begin{bmatrix}0\\ 0\end{bmatrix} + \begin{bmatrix}0.25\\ 0.09\end{bmatrix} = \begin{bmatrix}0.25\\ 0.09\end{bmatrix} \tag{176}$$

$$D_1 = \text{diag}\left(\frac{1}{\sqrt{0.25}}, \frac{1}{\sqrt{0.09}}\right) = \begin{bmatrix}2.0 & 0\\ 0 & 3.33\end{bmatrix} \tag{177}$$

$$\theta_1 = \begin{bmatrix}1.0\\ 2.0\end{bmatrix} - 0.1 \begin{bmatrix}2.0 & 0\\ 0 & 3.33\end{bmatrix} \begin{bmatrix}0.5\\ 0.3\end{bmatrix} \tag{178}$$

$$= \begin{bmatrix}1.0\\ 2.0\end{bmatrix} - \begin{bmatrix}0.1\\ 0.1\end{bmatrix} = \begin{bmatrix}0.9\\ 1.9\end{bmatrix} \tag{179}$$

Step 2:

$$G_2 = \begin{bmatrix}0.25\\0.09\end{bmatrix} + \begin{bmatrix}0.16\\0.36\end{bmatrix} = \begin{bmatrix}0.41\\0.45\end{bmatrix} \tag{180}$$

$$D_2 = \begin{bmatrix}1.56 & 0\\0 & 1.49\end{bmatrix} \tag{181}$$

$$\theta_2 = \begin{bmatrix}0.9\\1.9\end{bmatrix} - 0.1\begin{bmatrix}1.56 & 0\\0 & 1.49\end{bmatrix}\begin{bmatrix}0.4\\0.6\end{bmatrix} \tag{182}$$

$$= \begin{bmatrix}0.84\\1.81\end{bmatrix} \tag{183}$$

The diagonal matrix adapts learning rates per parameter based on gradient history.

3. Eigenvalue Matrix in Diagonalization:

When a matrix A is diagonalizable:

$$A = Q\Lambda Q^{-1} \tag{184}$$

where Λ is a diagonal matrix of eigenvalues.

Example:

$A = \begin{bmatrix}4 & 1\\2 & 3\end{bmatrix}$

Eigenvalues:

$$\det(A - \lambda I) = (4-\lambda)(3-\lambda) - 2 = 0 \tag{185}$$

$$\lambda^2 - 7\lambda + 10 = 0 \tag{186}$$

$$\lambda_1 = 5, \quad \lambda_2 = 2 \tag{187}$$

Eigenvalue matrix:

$$\Lambda = \begin{bmatrix}5 & 0\\0 & 2\end{bmatrix} \tag{188}$$

Eigenvectors:

For $\lambda_1 = 5$: $v_1 = \begin{bmatrix}1\\1\end{bmatrix}$

For $\lambda_2 = 2$: $v_2 = \begin{bmatrix}-1\\2\end{bmatrix}$

Eigenvector matrix:

$$Q = \begin{bmatrix}1 & -1\\1 & 2\end{bmatrix}, \quad Q^{-1} = \frac{1}{3}\begin{bmatrix}2 & 1\\-1 & 1\end{bmatrix} \tag{189}$$

Verification:

$$Q\Lambda Q^{-1} = \begin{bmatrix}1 & -1\\1 & 2\end{bmatrix}\begin{bmatrix}5 & 0\\0 & 2\end{bmatrix}\frac{1}{3}\begin{bmatrix}2 & 1\\-1 & 1\end{bmatrix} \tag{190}$$

$$= \frac{1}{3}\begin{bmatrix}5 & -2\\5 & 4\end{bmatrix}\begin{bmatrix}2 & 1\\-1 & 1\end{bmatrix} \tag{191}$$

$$= \frac{1}{3}\begin{bmatrix}12 & 3\\6 & 9\end{bmatrix} = \begin{bmatrix}4 & 1\\2 & 3\end{bmatrix} = A \tag{192}$$

Application - Matrix Powers:
To compute A^{100}:

$$A^{100} = (Q\Lambda Q^{-1})^{100} = Q\Lambda^{100}Q^{-1} \tag{193}$$

$$\Lambda^{100} = \begin{bmatrix} 5^{100} & 0 \\ 0 & 2^{100} \end{bmatrix} \tag{194}$$

Much easier to compute powers of diagonal matrix!

4. Attention Mechanism Scaling in Transformers:
In transformers, scaled dot-product attention uses diagonal scaling:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{195}$$

The scaling factor $\frac{1}{\sqrt{d_k}}$ can be viewed as multiplication by diagonal matrix:

$$D = \frac{1}{\sqrt{d_k}} I_{d_k} \tag{196}$$

Example with $d_k = 4$:
Dot products before scaling: $\begin{bmatrix} 10 & 15 & 8 & 12 \end{bmatrix}$
Scaling matrix: $D = \frac{1}{2}I_4 = \text{diag}(0.5, 0.5, 0.5, 0.5)$
After scaling: $\begin{bmatrix} 5 & 7.5 & 4 & 6 \end{bmatrix}$
This prevents dot products from growing too large, which would cause gradients to vanish in softmax.

6 Matrix Equality

6.1 Definition

Definition: Two matrices A and B are equal if and only if:

1. Same Dimensions: Both matrices must have the same number of rows and columns.

2. Equal Elements: All corresponding elements must be equal.

Formally, $A = B$ if and only if:

$$a_{ij} = b_{ij} \quad \forall i, j \tag{197}$$

6.2 Examples

Example 1 - Equal Matrices:

$$A = \begin{bmatrix} 2 & 3 & 5 \\ 7 & 1 & 9 \end{bmatrix} \tag{198}$$

$$B = \begin{bmatrix} 2 & 3 & 5 \\ 7 & 1 & 9 \end{bmatrix} \tag{199}$$

$A = B$ (same dimensions: 2×3, all elements equal)

Example 2 - Not Equal (Different Elements):

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \tag{200}$$

$$B = \begin{bmatrix} 1 & 2 \\ 3 & 5 \end{bmatrix} \tag{201}$$

$A \neq B$ because $a_{22} = 4 \neq 5 = b_{22}$

Example 3 - Not Equal (Different Dimensions):

$$A = \begin{bmatrix} 1 & 2 & 3 \end{bmatrix}_{1\times 3} \tag{202}$$

$$B = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}_{3\times 1} \tag{203}$$

$A \neq B$ because dimensions differ (even though they contain same elements)

6.3 Use in Machine Learning

1. Checking Algorithm Convergence:

In iterative algorithms, we check if parameter updates have converged by comparing matrices.

Example - Gradient Descent Convergence:

Weight matrix at iteration t: W_t

Weight matrix at iteration $t + 1$: W_{t+1}

Convergence criterion:

$$||W_{t+1} - W_t||_F < \epsilon \tag{204}$$

where $||\cdot||_F$ is the Frobenius norm and ϵ is tolerance (e.g., 10^{-6}).

Complete Example:

$$W_t = \begin{bmatrix} 0.523 & 0.891 \\ 0.345 & 0.678 \end{bmatrix} \tag{205}$$

$$W_{t+1} = \begin{bmatrix} 0.524 & 0.892 \\ 0.346 & 0.679 \end{bmatrix} \tag{206}$$

Calculate difference:

$$W_{t+1} - W_t = \begin{bmatrix} 0.001 & 0.001 \\ 0.001 & 0.001 \end{bmatrix} \tag{207}$$

Frobenius norm:

$$||W_{t+1} - W_t||_F = \sqrt{0.001^2 + 0.001^2 + 0.001^2 + 0.001^2} \tag{208}$$

$$= \sqrt{4 \times 10^{-6}} \tag{209}$$

$$= 0.002 \tag{210}$$

If $\epsilon = 0.01$, then $0.002 < 0.01 \rightarrow$ converged

2. Verifying Backpropagation Implementation:

Gradient checking compares numerical gradients with analytical gradients.

Numerical Gradient:

$$\frac{\partial L}{\partial W_{ij}} \approx \frac{L(W + \epsilon e_{ij}) - L(W - \epsilon e_{ij})}{2\epsilon} \tag{211}$$

where e_{ij} is a matrix with 1 at position (i, j) and 0 elsewhere.

Example:

Loss function: $L(W) = ||WX - Y||^2$

Given: $X = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$, $Y = \begin{bmatrix} 5 \\ 7 \end{bmatrix}$, $W = \begin{bmatrix} 2 & 1 \end{bmatrix}$

Analytical gradient:

$$\frac{\partial L}{\partial W} = 2(WX - Y)X^T \tag{212}$$

$$= 2\left(\begin{bmatrix} 2 & 1 \end{bmatrix}\begin{bmatrix} 1 \\ 2 \end{bmatrix} - \begin{bmatrix} 5 \\ 7 \end{bmatrix}\right)\begin{bmatrix} 1 & 2 \end{bmatrix} \tag{213}$$

$$= 2([4] - [5])\begin{bmatrix} 1 & 2 \end{bmatrix} \tag{214}$$

$$= 2(-1)\begin{bmatrix} 1 & 2 \end{bmatrix} = \begin{bmatrix} -2 & -4 \end{bmatrix} \tag{215}$$

Numerical gradient for W_{11}:

$\epsilon = 10^{-5}$

$W^+ = \begin{bmatrix} 2.00001 & 1 \end{bmatrix}$, $W^- = \begin{bmatrix} 1.99999 & 1 \end{bmatrix}$

$$L(W^+) = (2.00001 \cdot 1 + 1 \cdot 2 - 5)^2 = (-0.99999)^2 = 0.99998 \tag{216}$$

$$L(W^-) = (1.99999 \cdot 1 + 1 \cdot 2 - 5)^2 = (-1.00001)^2 = 1.00002 \tag{217}$$

$$\frac{\partial L}{\partial W_{11}} \approx \frac{0.99998 - 1.00002}{2 \times 10^{-5}} = \frac{-0.00004}{0.00002} = -2.0 \tag{218}$$

Analytical: -2, Numerical: $-2.0 \rightarrow$ Match!

If gradients don't match, there's a bug in backpropagation implementation.

3. Comparing Model Predictions:

When ensembling models or comparing architectures, we check if prediction matrices are equal or similar.

Example - Model Comparison:

Model A predictions: $\hat{Y}_A = \begin{bmatrix} 0.2 & 0.8 \\ 0.9 & 0.1 \\ 0.3 & 0.7 \end{bmatrix}$

Model B predictions: $\hat{Y}_B = \begin{bmatrix} 0.3 & 0.7 \\ 0.85 & 0.15 \\ 0.4 & 0.6 \end{bmatrix}$

Element-wise differences:

$$\hat{Y}_A - \hat{Y}_B = \begin{bmatrix} -0.1 & 0.1 \\ 0.05 & -0.05 \\ -0.1 & 0.1 \end{bmatrix} \tag{219}$$

Mean Absolute Difference:

$$\text{MAD} = \frac{1}{6} \sum |differences| = \frac{0.5}{6} \approx 0.083 \tag{220}$$

Models have 8.3% average difference in predictions.

4. Weight Sharing Verification in CNNs:

In convolutional layers, we verify that shared weights are actually equal across spatial locations.

Example:

Convolution kernel: $K = \begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix}$

This same kernel should be applied at all spatial positions. We verify:

$$K_{\text{position1}} = K_{\text{position2}} = \cdots = K \tag{221}$$

If any kernel differs, weight sharing is broken (implementation bug).

Practical Check: "'python Pseudo-code assert np.array$_equal(kernel_at_pos1, kernel_at_pos2)$"'

This ensures parameter sharing works correctly, reducing model parameters and improving generalization.

7 Summary

This comprehensive guide covered matrices in machine learning and deep learning:

- **Matrix Fundamentals:** Definition, notation, and representation of datasets, parameters, and transformations
- **Identity Matrices:** Multiplicative identity, use in regularization and linear algebra
- **Zero Matrices:** Initialization, padding, and masking in neural networks
- **Sparse Matrices:** Efficient representation for text data, recommendation systems, and pruned networks
- **Orthogonal Matrices:** PCA, SVD, weight initialization, and gradient flow preservation
- **Symmetric Matrices:** Covariance matrices, kernel matrices, Hessians, and graph Laplacians
- **Diagonal Matrices:** Feature scaling, adaptive learning rates, eigenvalue matrices, and attention scaling
- **Matrix Equality:** Convergence checking, gradient verification, and model comparison

All examples included complete calculations, verifications, and practical ML/DL applications.

8 Scalar Operations on Matrices

8.1 Definition

Definition: A scalar operation on a matrix involves multiplying or adding a single number (scalar) to every element of the matrix. These operations adjust all elements uniformly.

Scalar Multiplication: For a matrix A and scalar c:

$$cA = \begin{bmatrix} ca_{11} & ca_{12} & \cdots & ca_{1n} \\ ca_{21} & ca_{22} & \cdots & ca_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ ca_{m1} & ca_{m2} & \cdots & ca_{mn} \end{bmatrix} \tag{222}$$

Scalar Addition: For a matrix A and scalar c:

$$A + c = \begin{bmatrix} a_{11} + c & a_{12} + c & \cdots & a_{1n} + c \\ a_{21} + c & a_{22} + c & \cdots & a_{2n} + c \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} + c & a_{m2} + c & \cdots & a_{mn} + c \end{bmatrix} \tag{223}$$

8.2 Properties of Scalar Operations

1. Distributive Property:

$$c(A + B) = cA + cB \tag{224}$$

2. Associative Property:

$$(cd)A = c(dA) \tag{225}$$

3. Identity:

$$1 \cdot A = A \tag{226}$$

4. Zero Property:

$$0 \cdot A = 0 \tag{227}$$

8.3 Examples and Calculations

Example 1 - Scalar Multiplication:

Given matrix $A = \begin{bmatrix} 2 & 4 & 6 \\ 8 & 10 & 12 \end{bmatrix}$ and scalar $c = 3$

Step-by-step calculation:

$$3A = 3 \times \begin{bmatrix} 2 & 4 & 6 \\ 8 & 10 & 12 \end{bmatrix} \tag{228}$$

$$= \begin{bmatrix} 3(2) & 3(4) & 3(6) \\ 3(8) & 3(10) & 3(12) \end{bmatrix} \tag{229}$$

$$= \begin{bmatrix} 6 & 12 & 18 \\ 24 & 30 & 36 \end{bmatrix} \tag{230}$$

Each element is multiplied by 3.

Example 2 - Scalar Addition:

Given matrix $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix}$ and scalar $c = 5$

Step-by-step calculation:

$$A + 5 = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix} + 5 \tag{231}$$

$$= \begin{bmatrix} 1+5 & 2+5 \\ 3+5 & 4+5 \\ 5+5 & 6+5 \end{bmatrix} \tag{232}$$

$$= \begin{bmatrix} 6 & 7 \\ 8 & 9 \\ 10 & 11 \end{bmatrix} \tag{233}$$

Example 3 - Negative Scalar:

Given matrix $A = \begin{bmatrix} 3 & -2 & 1 \\ 0 & 5 & -4 \end{bmatrix}$ and scalar $c = -2$

$$-2A = -2 \times \begin{bmatrix} 3 & -2 & 1 \\ 0 & 5 & -4 \end{bmatrix} \tag{234}$$

$$= \begin{bmatrix} -6 & 4 & -2 \\ 0 & -10 & 8 \end{bmatrix} \tag{235}$$

Signs flip and magnitudes double.

Example 4 - Fractional Scalar:

Given matrix $A = \begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix}$ and scalar $c = 0.5$

$$0.5A = \frac{1}{2} \times \begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix} \tag{236}$$

$$= \begin{bmatrix} 5 & 10 \\ 15 & 20 \end{bmatrix} \tag{237}$$

All elements are halved.

8.4 Use in Machine Learning

1. Learning Rate Scaling in Gradient Descent:

In gradient descent, we update weights by multiplying gradients by a learning rate (scalar):

$$W_{new} = W_{old} - \eta \nabla L(W) \tag{238}$$

where η is the learning rate scalar and $\nabla L(W)$ is the gradient matrix.

Complete Example:

Current weights: $W = \begin{bmatrix} 0.5 & 0.8 \\ 0.3 & 0.6 \end{bmatrix}$

Computed gradients: $\nabla L = \begin{bmatrix} 0.2 & 0.4 \\ 0.1 & 0.3 \end{bmatrix}$

Learning rate: $\eta = 0.1$

Step 1: Scale gradients by learning rate:

$$\eta \nabla L = 0.1 \times \begin{bmatrix} 0.2 & 0.4 \\ 0.1 & 0.3 \end{bmatrix} \tag{239}$$

$$= \begin{bmatrix} 0.02 & 0.04 \\ 0.01 & 0.03 \end{bmatrix} \tag{240}$$

Step 2: Update weights:

$$W_{new} = \begin{bmatrix} 0.5 & 0.8 \\ 0.3 & 0.6 \end{bmatrix} - \begin{bmatrix} 0.02 & 0.04 \\ 0.01 & 0.03 \end{bmatrix} \tag{241}$$

$$= \begin{bmatrix} 0.48 & 0.76 \\ 0.29 & 0.57 \end{bmatrix} \tag{242}$$

The learning rate scalar controls the step size of weight updates.

2. Feature Normalization - Standard Scaling:

Standardization involves scalar operations to normalize features:

$$X_{scaled} = \frac{X - \mu}{\sigma} \tag{243}$$

Complete Example:

Dataset: $X = \begin{bmatrix} 10 & 100 \\ 20 & 200 \\ 30 & 300 \\ 40 & 400 \end{bmatrix}$

Step 1: Calculate means:

$$\mu = \begin{bmatrix} \frac{10+20+30+40}{4} & \frac{100+200+300+400}{4} \end{bmatrix} = \begin{bmatrix} 25 & 250 \end{bmatrix} \tag{244}$$

Step 2: Calculate standard deviations:

$$\sigma_1 = \sqrt{\frac{(10-25)^2 + (20-25)^2 + (30-25)^2 + (40-25)^2}{4}} \tag{245}$$

$$= \sqrt{\frac{225 + 25 + 25 + 225}{4}} = \sqrt{125} \approx 11.18 \tag{246}$$

$$\sigma_2 = \sqrt{\frac{(100-250)^2 + \cdots + (400-250)^2}{4}} \approx 111.8 \tag{247}$$

Step 3: Subtract mean (scalar subtraction):

$$X - \mu = \begin{bmatrix} -15 & -150 \\ -5 & -50 \\ 5 & 50 \\ 15 & 150 \end{bmatrix} \tag{248}$$

Step 4: Divide by standard deviation (scalar division):
For first column: divide by 11.18

$$\begin{bmatrix} -15/11.18 \\ -5/11.18 \\ 5/11.18 \\ 15/11.18 \end{bmatrix} = \begin{bmatrix} -1.34 \\ -0.45 \\ 0.45 \\ 1.34 \end{bmatrix} \tag{249}$$

For second column: divide by 111.8

$$\begin{bmatrix} -150/111.8 \\ -50/111.8 \\ 50/111.8 \\ 150/111.8 \end{bmatrix} = \begin{bmatrix} -1.34 \\ -0.45 \\ 0.45 \\ 1.34 \end{bmatrix} \tag{250}$$

Final scaled data:

$$X_{scaled} = \begin{bmatrix} -1.34 & -1.34 \\ -0.45 & -0.45 \\ 0.45 & 0.45 \\ 1.34 & 1.34 \end{bmatrix} \tag{251}$$

Now both features have mean 0 and standard deviation 1.

3. Weight Decay (L2 Regularization):
Weight decay multiplies all weights by a scalar slightly less than 1:

$$W_{new} = (1 - \lambda)W_{old} - \eta \nabla L(W) \tag{252}$$

where λ is the decay rate.
Example:
Weights: $W = \begin{bmatrix} 2.0 & 1.5 \\ 3.0 & 2.5 \end{bmatrix}$, $\lambda = 0.01$
Apply weight decay:

$$(1 - \lambda)W = 0.99 \times \begin{bmatrix} 2.0 & 1.5 \\ 3.0 & 2.5 \end{bmatrix} \tag{253}$$

$$= \begin{bmatrix} 1.98 & 1.485 \\ 2.97 & 2.475 \end{bmatrix} \tag{254}$$

This shrinks weights toward zero, preventing overfitting.
4. Temperature Scaling in Softmax:
In neural networks, temperature T is a scalar that controls softmax sharpness:

$$\text{softmax}_T(z_i) = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}} \tag{255}$$

Example:
Logits: $z = \begin{bmatrix} 2.0 & 1.0 & 0.5 \end{bmatrix}$
Case 1: Standard softmax ($T = 1$):

$$e^z = \begin{bmatrix} e^{2.0} & e^{1.0} & e^{0.5} \end{bmatrix} = \begin{bmatrix} 7.39 & 2.72 & 1.65 \end{bmatrix} \tag{256}$$

$$\sum = 7.39 + 2.72 + 1.65 = 11.76 \tag{257}$$

$$\text{softmax}(z) = \begin{bmatrix} 0.628 & 0.231 & 0.140 \end{bmatrix} \tag{258}$$

Case 2: Temperature scaling ($T = 2$):
First scale logits: $z/T = \begin{bmatrix} 1.0 & 0.5 & 0.25 \end{bmatrix}$

$$e^{z/T} = \begin{bmatrix} 2.72 & 1.65 & 1.28 \end{bmatrix} \tag{259}$$

$$\sum = 5.65 \tag{260}$$

$$\text{softmax}_2(z) = \begin{bmatrix} 0.481 & 0.292 & 0.227 \end{bmatrix} \tag{261}$$

Higher temperature produces softer (more uniform) probabilities.
5. Momentum in Optimization:
Momentum uses scalar multiplication to accumulate gradients:

$$v_t = \beta v_{t-1} + \nabla L(W_t) \tag{262}$$

$$W_{t+1} = W_t - \eta v_t \tag{263}$$

where β is the momentum coefficient (typically 0.9).
Complete Example:

Initial velocity: $v_0 = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}$

Gradient at step 1: $\nabla L_1 = \begin{bmatrix} 0.5 & 0.3 \\ 0.4 & 0.2 \end{bmatrix}$

$\beta = 0.9$, $\eta = 0.1$

Step 1:

$$v_1 = 0.9 \times \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix} + \begin{bmatrix} 0.5 & 0.3 \\ 0.4 & 0.2 \end{bmatrix} \tag{264}$$

$$= \begin{bmatrix} 0.5 & 0.3 \\ 0.4 & 0.2 \end{bmatrix} \tag{265}$$

Gradient at step 2: $\nabla L_2 = \begin{bmatrix} 0.4 & 0.2 \\ 0.3 & 0.1 \end{bmatrix}$

Step 2:

$$v_2 = 0.9 \times \begin{bmatrix} 0.5 & 0.3 \\ 0.4 & 0.2 \end{bmatrix} + \begin{bmatrix} 0.4 & 0.2 \\ 0.3 & 0.1 \end{bmatrix} \tag{266}$$

$$= \begin{bmatrix} 0.45 & 0.27 \\ 0.36 & 0.18 \end{bmatrix} + \begin{bmatrix} 0.4 & 0.2 \\ 0.3 & 0.1 \end{bmatrix} \tag{267}$$

$$= \begin{bmatrix} 0.85 & 0.47 \\ 0.66 & 0.28 \end{bmatrix} \tag{268}$$

The scalar β weights past gradients, creating momentum.

9 Matrix Addition and Subtraction

9.1 Definition

Definition: Matrix addition and subtraction are element-wise operations performed on matrices of the same dimensions.

Matrix Addition: For matrices A and B of size $m \times n$:

$$C = A + B \implies c_{ij} = a_{ij} + b_{ij} \tag{269}$$

Matrix Subtraction:

$$C = A - B \implies c_{ij} = a_{ij} - b_{ij} \tag{270}$$

Requirement: Matrices must have identical dimensions ($m \times n$).

9.2 Properties

1. Commutative Property (Addition only):

$$A + B = B + A \tag{271}$$

2. Associative Property:

$$(A + B) + C = A + (B + C) \tag{272}$$

3. Identity Element:

$$A + 0 = A \tag{273}$$

4. Inverse Element:

$$A + (-A) = 0 \tag{274}$$

5. Subtraction as Addition:

$$A - B = A + (-B) \tag{275}$$

9.3 Examples and Calculations

Example 1 - Matrix Addition:

$$A = \begin{bmatrix} 2 & 3 & 1 \\ 5 & 7 & 4 \end{bmatrix}, \quad B = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \tag{276}$$

Step-by-step calculation:

$$A + B = \begin{bmatrix} 2+1 & 3+2 & 1+3 \\ 5+4 & 7+5 & 4+6 \end{bmatrix} \tag{277}$$

$$= \begin{bmatrix} 3 & 5 & 4 \\ 9 & 12 & 10 \end{bmatrix} \tag{278}$$

Element-by-element breakdown:

$$c_{11} = a_{11} + b_{11} = 2 + 1 = 3 \tag{279}$$
$$c_{12} = a_{12} + b_{12} = 3 + 2 = 5 \tag{280}$$
$$c_{13} = a_{13} + b_{13} = 1 + 3 = 4 \tag{281}$$
$$c_{21} = a_{21} + b_{21} = 5 + 4 = 9 \tag{282}$$
$$c_{22} = a_{22} + b_{22} = 7 + 5 = 12 \tag{283}$$
$$c_{23} = a_{23} + b_{23} = 4 + 6 = 10 \tag{284}$$

Example 2 - Matrix Subtraction:

$$A = \begin{bmatrix} 8 & 6 \\ 4 & 2 \\ 10 & 12 \end{bmatrix}, \quad B = \begin{bmatrix} 3 & 2 \\ 1 & 5 \\ 7 & 4 \end{bmatrix} \tag{285}$$

$$A - B = \begin{bmatrix} 8-3 & 6-2 \\ 4-1 & 2-5 \\ 10-7 & 12-4 \end{bmatrix} \tag{286}$$
$$= \begin{bmatrix} 5 & 4 \\ 3 & -3 \\ 3 & 8 \end{bmatrix} \tag{287}$$

Note: $c_{22} = 2 - 5 = -3$ (negative result is valid)

Example 3 - With Negative Numbers:

$$A = \begin{bmatrix} -2 & 5 \\ 3 & -4 \end{bmatrix}, \quad B = \begin{bmatrix} 4 & -3 \\ -1 & 6 \end{bmatrix} \tag{288}$$

$$A + B = \begin{bmatrix} -2+4 & 5+(-3) \\ 3+(-1) & -4+6 \end{bmatrix} \tag{289}$$
$$= \begin{bmatrix} 2 & 2 \\ 2 & 2 \end{bmatrix} \tag{290}$$

Example 4 - Dimension Mismatch (Invalid):

$$A = \begin{bmatrix} 1 & 2 & 3 \end{bmatrix}_{1\times 3}, \quad B = \begin{bmatrix} 4 & 5 \end{bmatrix}_{1\times 2} \tag{291}$$

$A + B$ is **undefined** because dimensions don't match ($1 \times 3 \neq 1 \times 2$).

Example 5 - Verification of Commutative Property:

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} \tag{292}$$

$$A + B = \begin{bmatrix} 6 & 8 \\ 10 & 12 \end{bmatrix} \tag{293}$$
$$B + A = \begin{bmatrix} 5+1 & 6+2 \\ 7+3 & 8+4 \end{bmatrix} = \begin{bmatrix} 6 & 8 \\ 10 & 12 \end{bmatrix} \tag{294}$$

Therefore: $A + B = B + A$

9.4 Use in Machine Learning

1. Combining Predictions from Multiple Models (Ensemble):
In ensemble learning, predictions from different models are combined:

$$Y_{ensemble} = \frac{1}{n}(Y_1 + Y_2 + \cdots + Y_n) \tag{295}$$

Complete Example:
Three models predict probabilities for 4 samples, 2 classes:

$$Y_1 = \begin{bmatrix} 0.7 & 0.3 \\ 0.2 & 0.8 \\ 0.6 & 0.4 \\ 0.9 & 0.1 \end{bmatrix} \tag{296}$$

$$Y_2 = \begin{bmatrix} 0.8 & 0.2 \\ 0.3 & 0.7 \\ 0.5 & 0.5 \\ 0.85 & 0.15 \end{bmatrix} \tag{297}$$

$$Y_3 = \begin{bmatrix} 0.75 & 0.25 \\ 0.25 & 0.75 \\ 0.55 & 0.45 \\ 0.9 & 0.1 \end{bmatrix} \tag{298}$$

Step 1: Add all predictions:

$$Y_1 + Y_2 + Y_3 = \begin{bmatrix} 0.7+0.8+0.75 & 0.3+0.2+0.25 \\ 0.2+0.3+0.25 & 0.8+0.7+0.75 \\ 0.6+0.5+0.55 & 0.4+0.5+0.45 \\ 0.9+0.85+0.9 & 0.1+0.15+0.1 \end{bmatrix} \tag{299}$$

$$= \begin{bmatrix} 2.25 & 0.75 \\ 0.75 & 2.25 \\ 1.65 & 1.35 \\ 2.65 & 0.35 \end{bmatrix} \tag{300}$$

Step 2: Divide by number of models (3):

$$Y_{ensemble} = \frac{1}{3}\begin{bmatrix} 2.25 & 0.75 \\ 0.75 & 2.25 \\ 1.65 & 1.35 \\ 2.65 & 0.35 \end{bmatrix} = \begin{bmatrix} 0.75 & 0.25 \\ 0.25 & 0.75 \\ 0.55 & 0.45 \\ 0.883 & 0.117 \end{bmatrix} \tag{301}$$

The ensemble prediction is more robust than individual models.
2. Residual Connections in Neural Networks:
Residual blocks add the input directly to the output:

$$Y = F(X, W) + X \tag{302}$$

where $F(X, W)$ is the transformation and X is the input (skip connection).
Complete Example:

Input: $X = \begin{bmatrix} 1.0 & 0.5 \\ 0.8 & 0.3 \end{bmatrix}$

After transformation: $F(X, W) = \begin{bmatrix} 0.2 & 0.1 \\ -0.1 & 0.4 \end{bmatrix}$

Residual connection:

$$Y = F(X, W) + X \tag{303}$$

$$= \begin{bmatrix} 0.2 & 0.1 \\ -0.1 & 0.4 \end{bmatrix} + \begin{bmatrix} 1.0 & 0.5 \\ 0.8 & 0.3 \end{bmatrix} \tag{304}$$

$$= \begin{bmatrix} 1.2 & 0.6 \\ 0.7 & 0.7 \end{bmatrix} \tag{305}$$

This helps gradients flow backward during training (solves vanishing gradient problem).

3. Computing Prediction Errors:

Mean Squared Error involves matrix subtraction:

$$MSE = \frac{1}{n}||Y_{true} - Y_{pred}||^2 \tag{306}$$

Complete Example:

True labels: $Y_{true} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \end{bmatrix}$

Predictions: $Y_{pred} = \begin{bmatrix} 0.9 & 0.1 \\ 0.2 & 0.8 \\ 0.85 & 0.15 \end{bmatrix}$

Step 1: Compute errors:

$$E = Y_{true} - Y_{pred} \tag{307}$$

$$= \begin{bmatrix} 1 - 0.9 & 0 - 0.1 \\ 0 - 0.2 & 1 - 0.8 \\ 1 - 0.85 & 0 - 0.15 \end{bmatrix} \tag{308}$$

$$= \begin{bmatrix} 0.1 & -0.1 \\ -0.2 & 0.2 \\ 0.15 & -0.15 \end{bmatrix} \tag{309}$$

Step 2: Square each element:

$$E^2 = \begin{bmatrix} 0.01 & 0.01 \\ 0.04 & 0.04 \\ 0.0225 & 0.0225 \end{bmatrix} \tag{310}$$

Step 3: Calculate MSE:

$$MSE = \frac{1}{6}(0.01 + 0.01 + 0.04 + 0.04 + 0.0225 + 0.0225) = \frac{0.145}{6} \approx 0.024 \tag{311}$$

4. Batch Normalization - Centering:

Batch normalization subtracts the batch mean from features:

$$\hat{X} = X - \mu_B \tag{312}$$

Complete Example:

Batch data: $X = \begin{bmatrix} 2 & 4 \\ 4 & 6 \\ 6 & 8 \\ 8 & 10 \end{bmatrix}$

Step 1: Compute batch mean:

$$\mu_B = \begin{bmatrix} \frac{2+4+6+8}{4} & \frac{4+6+8+10}{4} \end{bmatrix} = \begin{bmatrix} 5 & 7 \end{bmatrix} \tag{313}$$

Step 2: Center the data:

$$\hat{X} = X - \mu_B \tag{314}$$

$$= \begin{bmatrix} 2 & 4 \\ 4 & 6 \\ 6 & 8 \\ 8 & 10 \end{bmatrix} - \begin{bmatrix} 5 & 7 \end{bmatrix} \tag{315}$$

$$= \begin{bmatrix} -3 & -3 \\ -1 & -1 \\ 1 & 1 \\ 3 & 3 \end{bmatrix} \tag{316}$$

Centered data has mean zero, stabilizing training.

5. Gradient Accumulation:

In mini-batch training with gradient accumulation:

$$\nabla L_{total} =_1 + \nabla L_2 + \cdots + \nabla L_k \tag{317}$$

Example:

Gradients from 3 mini-batches:

$$\nabla L_1 = \begin{bmatrix} 0.1 & 0.2 \\ 0.3 & 0.4 \end{bmatrix} \tag{318}$$

$$\nabla L_2 = \begin{bmatrix} 0.15 & 0.25 \\ 0.35 & 0.45 \end{bmatrix} \tag{319}$$

$$\nabla L_3 = \begin{bmatrix} 0.12 & 0.22 \\ 0.32 & 0.42 \end{bmatrix} \tag{320}$$

Accumulated gradient:

$$\nabla L_{total} = \nabla L_1 + \nabla L_2 + \nabla L_3 \tag{321}$$

$$= \begin{bmatrix} 0.1 + 0.15 + 0.12 & 0.2 + 0.25 + 0.22 \\ 0.3 + 0.35 + 0.32 & 0.4 + 0.45 + 0.42 \end{bmatrix} \tag{322}$$

$$= \begin{bmatrix} 0.37 & 0.67 \\ 0.97 & 1.27 \end{bmatrix} \tag{323}$$

This allows training with effectively larger batch sizes.

10 Matrix Multiplication

10.1 Definition

Definition: Matrix multiplication is a binary operation that produces a matrix from two matrices. For matrices A of size $m \times n$ and B of size $n \times p$, the product $C = AB$ is of size $m \times p$.

Formula: Each element c_{ij} is computed as:

$$c_{ij} = \sum_{k=1}^{n} a_{ik} b_{kj} = a_{i1} b_{1j} + a_{i2} b_{2j} + \cdots + a_{in} b_{nj} \tag{324}$$

Dimension Requirement: The number of columns in A must equal the number of rows in B.

Notation:

$$\underbrace{A}_{m \times n} \cdot \underbrace{B}_{n \times p} = \underbrace{C}_{m \times p} \tag{325}$$

10.2 Properties

1. Not Commutative:

$$AB \neq BA \quad \text{(in general)} \tag{326}$$

2. Associative:

$$(AB)C = A(BC) \tag{327}$$

3. Distributive:

$$A(B + C) = AB + AC \tag{328}$$
$$(A + B)C = AC + BC \tag{329}$$

4. Identity:

$$AI = IA = A \tag{330}$$

5. Transpose Property:

$$(AB)^T = B^T A^T \tag{331}$$

10.3 Examples and Calculations

Example 1 - Basic 2×2 Multiplication:

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} \tag{332}$$

Detailed calculation:

$$c_{11} = a_{11} b_{11} + a_{12} b_{21} = 1(5) + 2(7) = 5 + 14 = 19 \tag{333}$$
$$c_{12} = a_{11} b_{12} + a_{12} b_{22} = 1(6) + 2(8) = 6 + 16 = 22 \tag{334}$$
$$c_{21} = a_{21} b_{11} + a_{22} b_{21} = 3(5) + 4(7) = 15 + 28 = 43 \tag{335}$$
$$c_{22} = a_{21} b_{12} + a_{22} b_{22} = 3(6) + 4(8) = 18 + 32 = 50 \tag{336}$$

Result:

$$AB = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix} \tag{337}$$

Verification of non-commutativity:

$$BA = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \tag{338}$$

$$= \begin{bmatrix} 5(1)+6(3) & 5(2)+6(4) \\ 7(1)+8(3) & 7(2)+8(4) \end{bmatrix} \tag{339}$$

$$= \begin{bmatrix} 23 & 34 \\ 31 & 46 \end{bmatrix} \neq AB \tag{340}$$

Example 2 - Non-Square Matrices:

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}_{2\times 3}, \quad B = \begin{bmatrix} 7 & 8 \\ 9 & 10 \\ 11 & 12 \end{bmatrix}_{3\times 2} \tag{341}$$

Dimensions check: A is 2×3 and B is 3×2, so AB will be 2×2
Detailed calculation:

$$c_{11} = 1(7) + 2(9) + 3(11) = 7 + 18 + 33 = 58 \tag{342}$$

$$c_{12} = 1(8) + 2(10) + 3(12) = 8 + 20 + 36 = 64 \tag{343}$$

$$c_{21} = 4(7) + 5(9) + 6(11) = 28 + 45 + 66 = 139 \tag{344}$$

$$c_{22} = 4(8) + 5(10) + 6(12) = 32 + 50 + 72 = 154 \tag{345}$$

Result:

$$AB = \begin{bmatrix} 58 & 64 \\ 139 & 154 \end{bmatrix}_{2\times 2} \tag{346}$$

Example 3 - Matrix-Vector Multiplication:

$$A = \begin{bmatrix} 2 & 3 & 1 \\ 4 & 0 & 2 \\ 1 & 5 & 3 \end{bmatrix}, \quad x = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} \tag{347}$$

Calculation:

$$Ax = \begin{bmatrix} 2 & 3 & 1 \\ 4 & 0 & 2 \\ 1 & 5 & 3 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} \tag{348}$$

$$= \begin{bmatrix} 2(1)+3(2)+1(3) \\ 4(1)+0(2)+2(3) \\ 1(1)+5(2)+3(3) \end{bmatrix} \tag{349}$$

$$= \begin{bmatrix} 2+6+3 \\ 4+0+6 \\ 1+10+9 \end{bmatrix} \tag{350}$$

$$= \begin{bmatrix} 11 \\ 10 \\ 20 \end{bmatrix} \tag{351}$$

Example 4 - Invalid Multiplication:

$$A = \begin{bmatrix} 1 & 2 \end{bmatrix}_{1\times 2}, \quad B = \begin{bmatrix} 3 & 4 & 5 \end{bmatrix}_{1\times 3} \tag{352}$$

AB is **undefined** because the number of columns in A (2) $\neq$ number of rows in B (1).

Example 5 - Identity Matrix Verification:

$$A = \begin{bmatrix} 3 & 5 \\ 7 & 9 \end{bmatrix}, \quad I = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \tag{353}$$

$$AI = \begin{bmatrix} 3 & 5 \\ 7 & 9 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \tag{354}$$

$$= \begin{bmatrix} 3(1)+5(0) & 3(0)+5(1) \\ 7(1)+9(0) & 7(0)+9(1) \end{bmatrix} \tag{355}$$

$$= \begin{bmatrix} 3 & 5 \\ 7 & 9 \end{bmatrix} = A \tag{356}$$

10.4 Use in Machine Learning

1. Linear Regression - Computing Predictions:

Linear regression predictions: $\hat{y} = Xw$

Complete Example:

Feature matrix (5 samples, 2 features + bias):

$$X = \begin{bmatrix} 1 & 1500 & 3 \\ 1 & 2000 & 4 \\ 1 & 1200 & 2 \\ 1 & 1800 & 3 \\ 1 & 2200 & 4 \end{bmatrix} \tag{357}$$

Weights: $w = \begin{bmatrix} 50000 \\ 100 \\ 10000 \end{bmatrix}$ (intercept, area weight, bedroom weight)

Calculate predictions:

$$\hat{y} = Xw = \begin{bmatrix} 1 & 1500 & 3 \\ 1 & 2000 & 4 \\ 1 & 1200 & 2 \\ 1 & 1800 & 3 \\ 1 & 2200 & 4 \end{bmatrix} \begin{bmatrix} 50000 \\ 100 \\ 10000 \end{bmatrix} \tag{358}$$

Detailed calculation for each sample:

$$\hat{y}_1 = 1(50000) + 1500(100) + 3(10000) = 50000 + 150000 + 30000 = 230000 \tag{359}$$

$$\hat{y}_2 = 1(50000) + 2000(100) + 4(10000) = 50000 + 200000 + 40000 = 290000 \tag{360}$$

$$\hat{y}_3 = 1(50000) + 1200(100) + 2(10000) = 50000 + 120000 + 20000 = 190000 \tag{361}$$

$$\hat{y}_4 = 1(50000) + 1800(100) + 3(10000) = 50000 + 180000 + 30000 = 260000 \tag{362}$$

$$\hat{y}_5 = 1(50000) + 2200(100) + 4(10000) = 50000 + 220000 + 40000 = 310000 \tag{363}$$

Result:

$$\hat{y} = \begin{bmatrix} 230000 \\ 290000 \\ 190000 \\ 260000 \\ 310000 \end{bmatrix} \tag{364}$$

These are the predicted house prices.

2. Forward Propagation in Neural Networks:

In a neural network layer: $h = \sigma(Wx + b)$

Complete Example:

Input: $x = \begin{bmatrix} 0.5 \\ 0.3 \\ 0.8 \end{bmatrix}$ (3 features)

Weights: $W = \begin{bmatrix} 0.2 & 0.4 & -0.1 \\ 0.5 & -0.3 & 0.6 \end{bmatrix}$ (2 neurons, 3 inputs)

Bias: $b = \begin{bmatrix} 0.1 \\ -0.2 \end{bmatrix}$

Step 1: Matrix multiplication Wx:

$$Wx = \begin{bmatrix} 0.2 & 0.4 & -0.1 \\ 0.5 & -0.3 & 0.6 \end{bmatrix} \begin{bmatrix} 0.5 \\ 0.3 \\ 0.8 \end{bmatrix} \tag{365}$$

$$= \begin{bmatrix} 0.2(0.5) + 0.4(0.3) + (-0.1)(0.8) \\ 0.5(0.5) + (-0.3)(0.3) + 0.6(0.8) \end{bmatrix} \tag{366}$$

$$= \begin{bmatrix} 0.1 + 0.12 - 0.08 \\ 0.25 - 0.09 + 0.48 \end{bmatrix} \tag{367}$$

$$= \begin{bmatrix} 0.14 \\ 0.64 \end{bmatrix} \tag{368}$$

Step 2: Add bias:

$$Wx + b = \begin{bmatrix} 0.14 \\ 0.64 \end{bmatrix} + \begin{bmatrix} 0.1 \\ -0.2 \end{bmatrix} = \begin{bmatrix} 0.24 \\ 0.44 \end{bmatrix} \tag{369}$$

Step 3: Apply activation (sigmoid): $\sigma(z) = \frac{1}{1+e^{-z}}$:

$$h_1 = \frac{1}{1 + e^{-0.24}} = \frac{1}{1.271} = 0.560 \tag{370}$$

$$h_2 = \frac{1}{1 + e^{-0.44}} = \frac{1}{1.553} = 0.608 \tag{371}$$

Output:

$$h = \begin{bmatrix} 0.560 \\ 0.608 \end{bmatrix} \tag{372}$$

3. Batch Processing:

Processing multiple samples simultaneously:

Input batch (4 samples, 3 features):

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \\ 10 & 11 & 12 \end{bmatrix} \tag{373}$$

Weights (3 features, 2 outputs):

$$W = \begin{bmatrix} 0.1 & 0.2 \\ 0.3 & 0.4 \\ 0.5 & 0.6 \end{bmatrix} \tag{374}$$

Batch computation:

$$Y = XW \tag{375}$$

$$= \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \\ 10 & 11 & 12 \end{bmatrix} \begin{bmatrix} 0.1 & 0.2 \\ 0.3 & 0.4 \\ 0.5 & 0.6 \end{bmatrix} \tag{376}$$

Calculation for first row:

$$y_{11} = 1(0.1) + 2(0.3) + 3(0.5) = 0.1 + 0.6 + 1.5 = 2.2 \tag{377}$$

$$y_{12} = 1(0.2) + 2(0.4) + 3(0.6) = 0.2 + 0.8 + 1.8 = 2.8 \tag{378}$$

Calculation for second row:

$$y_{21} = 4(0.1) + 5(0.3) + 6(0.5) = 0.4 + 1.5 + 3.0 = 4.9 \tag{379}$$

$$y_{22} = 4(0.2) + 5(0.4) + 6(0.6) = 0.8 + 2.0 + 3.6 = 6.4 \tag{380}$$

Calculation for third row:

$$y_{31} = 7(0.1) + 8(0.3) + 9(0.5) = 0.7 + 2.4 + 4.5 = 7.6 \tag{381}$$

$$y_{32} = 7(0.2) + 8(0.4) + 9(0.6) = 1.4 + 3.2 + 5.4 = 10.0 \tag{382}$$

Calculation for fourth row:

$$y_{41} = 10(0.1) + 11(0.3) + 12(0.5) = 1.0 + 3.3 + 6.0 = 10.3 \tag{383}$$

$$y_{42} = 10(0.2) + 11(0.4) + 12(0.6) = 2.0 + 4.4 + 7.2 = 13.6 \tag{384}$$

Result:

$$Y = \begin{bmatrix} 2.2 & 2.8 \\ 4.9 & 6.4 \\ 7.6 & 10.0 \\ 10.3 & 13.6 \end{bmatrix} \tag{385}$$

All 4 samples processed in one operation!

4. Attention Mechanism in Transformers:

Attention: $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$

Example:

Query: $Q = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$, Key: $K = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$, Value: $V = \begin{bmatrix} 2 & 1 \\ 1 & 3 \end{bmatrix}$

Step 1: Compute QK^T:

$$K^T = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix} \tag{386}$$

$$QK^T = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix} \tag{387}$$

Step 2: Scale by $\sqrt{d_k} = \sqrt{2}$:

$$\frac{QK^T}{\sqrt{2}} = \begin{bmatrix} 0.707 & 0 \\ 0.707 & 0.707 \end{bmatrix} \tag{388}$$

Step 3: Apply softmax (row-wise):

$$\text{Row 1: } e^{0.707} = 2.028, \quad e^0 = 1.000, \quad \text{sum} = 3.028 \tag{389}$$

$$\text{softmax} = \begin{bmatrix} 0.670 & 0.330 \end{bmatrix} \tag{390}$$

$$\text{Row 2: } e^{0.707} = 2.028, \quad e^{0.707} = 2.028, \quad \text{sum} = 4.056 \tag{391}$$

$$\text{softmax} = \begin{bmatrix} 0.500 & 0.500 \end{bmatrix} \tag{392}$$

Attention weights:

$$A = \begin{bmatrix} 0.670 & 0.330 \\ 0.500 & 0.500 \end{bmatrix} \tag{393}$$

Step 4: Multiply by values:

$$\text{Output} = AV = \begin{bmatrix} 0.670 & 0.330 \\ 0.500 & 0.500 \end{bmatrix} \begin{bmatrix} 2 & 1 \\ 1 & 3 \end{bmatrix} \tag{394}$$

$$= \begin{bmatrix} 0.670(2) + 0.330(1) & 0.670(1) + 0.330(3) \\ 0.500(2) + 0.500(1) & 0.500(1) + 0.500(3) \end{bmatrix} \tag{395}$$

$$= \begin{bmatrix} 1.67 & 1.66 \\ 1.50 & 2.00 \end{bmatrix} \tag{396}$$

5. Computing Gram Matrix for Style Transfer:
Gram matrix: $G = FF^T$ where F is feature matrix.
Example:
Feature map: $F = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix}$ (3 positions, 2 channels)

$$G = FF^T = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix} \begin{bmatrix} 1 & 3 & 5 \\ 2 & 4 & 6 \end{bmatrix} \tag{397}$$

$$= \begin{bmatrix} 1(1) + 2(2) & 1(3) + 2(4) & 1(5) + 2(6) \\ 3(1) + 4(2) & 3(3) + 4(4) & 3(5) + 4(6) \\ 5(1) + 6(2) & 5(3) + 6(4) & 5(5) + 6(6) \end{bmatrix} \tag{398}$$

$$= \begin{bmatrix} 5 & 11 & 17 \\ 11 & 25 & 39 \\ 17 & 39 & 61 \end{bmatrix} \tag{399}$$

This captures style correlations between feature channels.

11 Transpose of a Matrix

11.1 Definition

Definition: The transpose of a matrix A is obtained by flipping the matrix over its diagonal, converting rows to columns and columns to rows. Denoted as A^T.

Formal Definition: If A is an $m \times n$ matrix, then A^T is an $n \times m$ matrix where:

$$(A^T)_{ij} = A_{ji} \tag{400}$$

Notation:

$$\text{If } A = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \end{bmatrix}_{2\times 3}, \text{ then } A^T = \begin{bmatrix} a_{11} & a_{21} \\ a_{12} & a_{22} \\ a_{13} & a_{23} \end{bmatrix}_{3\times 2} \tag{401}$$

11.2 Properties

1. Double Transpose:

$$(A^T)^T = A \tag{402}$$

2. Transpose of Sum:

$$(A + B)^T = A^T + B^T \tag{403}$$

3. Transpose of Product:

$$(AB)^T = B^T A^T \quad \text{(order reverses!)} \tag{404}$$

4. Transpose of Scalar Multiple:

$$(cA)^T = cA^T \tag{405}$$

5. Transpose of Inverse:

$$(A^{-1})^T = (A^T)^{-1} \tag{406}$$

11.3 Examples and Calculations

Example 1 - Basic Transpose:

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}_{2\times 3} \tag{407}$$

Transpose:

$$A^T = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}_{3\times 2} \tag{408}$$

Verification: Row 1 of A becomes column 1 of A^T, row 2 of A becomes column 2 of A^T

Example 2 - Square Matrix Transpose:

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} \tag{409}$$

Transpose:

$$A^T = \begin{bmatrix} 1 & 4 & 7 \\ 2 & 5 & 8 \\ 3 & 6 & 9 \end{bmatrix} \tag{410}$$

Elements along the main diagonal (1, 5, 9) remain in place.

Example 3 - Column Vector Transpose:

$$x = \begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \end{bmatrix}_{4\times 1} \tag{411}$$

Transpose (row vector):

$$x^T = \begin{bmatrix} 1 & 2 & 3 & 4 \end{bmatrix}_{1\times 4} \tag{412}$$

Example 4 - Symmetric Matrix:

$$A = \begin{bmatrix} 2 & 3 & 5 \\ 3 & 4 & 6 \\ 5 & 6 & 7 \end{bmatrix} \tag{413}$$

Transpose:

$$A^T = \begin{bmatrix} 2 & 3 & 5 \\ 3 & 4 & 6 \\ 5 & 6 & 7 \end{bmatrix} = A \tag{414}$$

Symmetric matrices satisfy $A = A^T$

Example 5 - Verifying Transpose of Product Property:

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} \tag{415}$$

Method 1: Compute AB, then transpose:

$$AB = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix} \tag{416}$$

$$(AB)^T = \begin{bmatrix} 19 & 43 \\ 22 & 50 \end{bmatrix} \tag{417}$$

Method 2: Compute $B^T A^T$:

$$A^T = \begin{bmatrix} 1 & 3 \\ 2 & 4 \end{bmatrix}, \quad B^T = \begin{bmatrix} 5 & 7 \\ 6 & 8 \end{bmatrix} \tag{418}$$

$$B^T A^T = \begin{bmatrix} 5 & 7 \\ 6 & 8 \end{bmatrix} \begin{bmatrix} 1 & 3 \\ 2 & 4 \end{bmatrix} \tag{419}$$

$$= \begin{bmatrix} 19 & 43 \\ 22 & 50 \end{bmatrix} \tag{420}$$

(420)

Using transpose:

$$x^T y = \begin{bmatrix} 2 & 3 & 4 \end{bmatrix} \begin{bmatrix} 1 \\ 5 \\ 2 \end{bmatrix} \tag{421}$$

$$= 2(1) + 3(5) + 4(2) \tag{422}$$

$$= 2 + 15 + 8 = 25 \tag{423}$$

This is essential for computing similarity between vectors in ML.

2. Normal Equations in Linear Regression:

The closed-form solution for linear regression uses transpose:

$$w = (X^T X)^{-1} X^T y \tag{424}$$

Complete Example:

Dataset: $X = \begin{bmatrix} 1 & 2 \\ 1 & 4 \\ 1 & 6 \end{bmatrix}$, $y = \begin{bmatrix} 3 \\ 5 \\ 7 \end{bmatrix}$

Step 1: Compute X^T:

$$X^T = \begin{bmatrix} 1 & 1 & 1 \\ 2 & 4 & 6 \end{bmatrix} \tag{425}$$

Step 2: Compute $X^T X$:

$$X^T X = \begin{bmatrix} 1 & 1 & 1 \\ 2 & 4 & 6 \end{bmatrix} \begin{bmatrix} 1 & 2 \\ 1 & 4 \\ 1 & 6 \end{bmatrix} \tag{426}$$

$$= \begin{bmatrix} 3 & 12 \\ 12 & 56 \end{bmatrix} \tag{427}$$

Step 3: Compute $(X^T X)^{-1}$:

$$\det(X^T X) = 3(56) - 12(12) = 168 - 144 = 24 \tag{428}$$

$$(X^T X)^{-1} = \frac{1}{24} \begin{bmatrix} 56 & -12 \\ -12 & 3 \end{bmatrix} = \begin{bmatrix} 2.33 & -0.5 \\ -0.5 & 0.125 \end{bmatrix} \tag{429}$$

Step 4: Compute $X^T y$:

$$X^T y = \begin{bmatrix} 1 & 1 & 1 \\ 2 & 4 & 6 \end{bmatrix} \begin{bmatrix} 3 \\ 5 \\ 7 \end{bmatrix} = \begin{bmatrix} 15 \\ 68 \end{bmatrix} \tag{430}$$

Step 5: Compute w:

$$w = \begin{bmatrix} 2.33 & -0.5 \\ -0.5 & 0.125 \end{bmatrix} \begin{bmatrix} 15 \\ 68 \end{bmatrix} \tag{431}$$

$$= \begin{bmatrix} 2.33(15) - 0.5(68) \\ -0.5(15) + 0.125(68) \end{bmatrix} \tag{432}$$

$$= \begin{bmatrix} 34.95 - 34 \\ -7.5 + 8.5 \end{bmatrix} = \begin{bmatrix} 0.95 \\ 1.0 \end{bmatrix} \tag{433}$$

Model: $\hat{y} = 0.95 + 1.0x$

3. Backpropagation - Gradient Computation:

Gradients with respect to weights involve transpose:

$$\frac{\partial L}{\partial W} = X^T \delta \tag{434}$$

where δ is the error signal.

Example:

Input: $X = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$, Error: $\delta = \begin{bmatrix} 0.1 & 0.2 \\ 0.3 & 0.4 \end{bmatrix}$

Compute gradient:

$$\frac{\partial L}{\partial W} = X^T \delta \tag{435}$$

$$= \begin{bmatrix} 1 & 3 \\ 2 & 4 \end{bmatrix} \begin{bmatrix} 0.1 & 0.2 \\ 0.3 & 0.4 \end{bmatrix} \tag{436}$$

$$= \begin{bmatrix} 1(0.1) + 3(0.3) & 1(0.2) + 3(0.4) \\ 2(0.1) + 4(0.3) & 2(0.2) + 4(0.4) \end{bmatrix} \tag{437}$$

$$= \begin{bmatrix} 1.0 & 1.4 \\ 1.4 & 2.0 \end{bmatrix} \tag{438}$$

This gradient is used to update weights.

4. Covariance Matrix Computation:

Covariance: $\Sigma = \frac{1}{n-1} X^T X$ (for centered data)

Example:

Centered data: $X = \begin{bmatrix} -1 & -2 \\ 0 & 0 \\ 1 & 2 \end{bmatrix}$

$$X^T X = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \end{bmatrix} \begin{bmatrix} -1 & -2 \\ 0 & 0 \\ 1 & 2 \end{bmatrix} \tag{439}$$

$$= \begin{bmatrix} 2 & 4 \\ 4 & 8 \end{bmatrix} \tag{440}$$

$$\Sigma = \frac{1}{2} \begin{bmatrix} 2 & 4 \\ 4 & 8 \end{bmatrix} = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix} \tag{441}$$

5. Self-Attention in Transformers:

Keys and queries interact via transpose:

$$\text{scores} = QK^T \tag{442}$$

Example:

Query: $Q = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}$, Key: $K = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \end{bmatrix}$

$$K^T = \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 0 & 1 \end{bmatrix} \tag{443}$$

$$QK^T = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 0 & 1 \end{bmatrix} \tag{444}$$

$$= \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \tag{445}$$

These scores determine attention weights between queries and keys.

12 Determinant

12.1 Definition

Definition: The determinant is a scalar value that can be computed from a square matrix. It provides important information about the matrix, including whether it's invertible and the volume scaling factor of the linear transformation it represents.

Notation: For matrix A, the determinant is denoted as:

$$\det(A) \text{ or } |A| \tag{446}$$

Geometric Interpretation: The absolute value of the determinant represents the volume scaling factor when the matrix transforms space.

12.2 Formulas for Different Sizes

1. For 1×1 Matrix:

$$A = [a] \implies \det(A) = a \tag{447}$$

2. For 2×2 Matrix:

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \implies \det(A) = ad - bc \tag{448}$$

3. For 3×3 Matrix (Rule of Sarrus):

$$A = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix} \tag{449}$$

$$\det(A) = aei + bfg + cdh - ceg - afh - bdi \tag{450}$$

4. General Formula (Cofactor Expansion):
For an $n \times n$ matrix, expand along row i:

$$\det(A) = \sum_{j=1}^{n} (-1)^{i+j} a_{ij} M_{ij} \tag{451}$$

where M_{ij} is the minor (determinant of the submatrix after removing row i and column j).

12.3 Properties

1. Determinant of Identity:

$$\det(I) = 1 \tag{452}$$

2. Determinant of Product:

$$\det(AB) = \det(A) \cdot \det(B) \tag{453}$$

3. Determinant of Transpose:

$$\det(A^T) = \det(A) \tag{454}$$

4. Determinant of Inverse:

$$\det(A^{-1}) = \frac{1}{\det(A)} \tag{455}$$

5. Scalar Multiplication:

$$\det(cA) = c^n \det(A) \quad \text{for } n \times n \text{ matrix} \tag{456}$$

6. Invertibility:

$$A \text{ is invertible} \iff \det(A) \neq 0 \tag{457}$$

12.4 Examples and Calculations

Example 1 - 2×2 Matrix:

$$A = \begin{bmatrix} 3 & 5 \\ 2 & 4 \end{bmatrix} \tag{458}$$

Calculation:

$$\begin{aligned} \det(A) &= (3)(4) - (5)(2) && (459)\\ &= 12 - 10 && (460)\\ &= 2 && (461) \end{aligned}$$

Since $\det(A) = 2 \neq 0$, matrix A is invertible

Example 2 - 3×3 Matrix:

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 0 & 4 & 5 \\ 1 & 0 & 6 \end{bmatrix} \tag{462}$$

Using cofactor expansion along first row:

$$\det(A) = 1 \cdot \begin{vmatrix} 4 & 5 \\ 0 & 6 \end{vmatrix} - 2 \cdot \begin{vmatrix} 0 & 5 \\ 1 & 6 \end{vmatrix} + 3 \cdot \begin{vmatrix} 0 & 4 \\ 1 & 0 \end{vmatrix} \tag{463}$$

Calculate each 2×2 determinant:

$$\begin{vmatrix} 4 & 5 \\ 0 & 6 \end{vmatrix} = 4(6) - 5(0) = 24 \tag{464}$$

$$\begin{vmatrix} 0 & 5 \\ 1 & 6 \end{vmatrix} = 0(6) - 5(1) = -5 \tag{465}$$

$$\begin{vmatrix} 0 & 4 \\ 1 & 0 \end{vmatrix} = 0(0) - 4(1) = -4 \tag{466}$$

Final calculation:

$$\begin{aligned} \det(A) &= 1(24) - 2(-5) + 3(-4) && (467)\\ &= 24 + 10 - 12 && (468)\\ &= 22 && (469) \end{aligned}$$

Example 3 - Diagonal Matrix:

$$D = \begin{bmatrix} 2 & 0 & 0 \\ 0 & 3 & 0 \\ 0 & 0 & 5 \end{bmatrix} \tag{470}$$

Property: Determinant of diagonal matrix is the product of diagonal elements:

$$\det(D) = 2 \times 3 \times 5 = 30 \tag{471}$$

Example 4 - Singular Matrix:

$$A = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix} \tag{472}$$

$$\det(A) = 1(4) - 2(2) \tag{473}$$
$$= 4 - 4 = 0 \tag{474}$$

Since $\det(A) = 0$, matrix A is **singular** (not invertible). Note: Row 2 is 2 times row 1, so rows are linearly dependent.

Example 5 - Product Property Verification:

$$A = \begin{bmatrix} 2 & 1 \\ 3 & 4 \end{bmatrix}, \quad B = \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix} \tag{475}$$

Determinants:

$$\det(A) = 2(4) - 1(3) = 8 - 3 = 5 \tag{476}$$
$$\det(B) = 1(1) - 2(0) = 1 \tag{477}$$

Product:

$$AB = \begin{bmatrix} 2 & 5 \\ 3 & 10 \end{bmatrix} \tag{478}$$

$$\det(AB) = 2(10) - 5(3) = 20 - 15 = 5 \tag{479}$$

Verification:

$$\det(A) \cdot \det(B) = 5 \cdot 1 = 5 = \det(AB) \tag{480}$$

12.5 Use in Machine Learning

1. Checking Matrix Invertibility:

In linear regression, $(X^T X)$ must be invertible:

Example:

$$X^T X = \begin{bmatrix} 10 & 5 \\ 5 & 3 \end{bmatrix} \tag{481}$$

$$\det(X^T X) = 10(3) - 5(5) \tag{482}$$
$$= 30 - 25 = 5 \tag{483}$$

Since $\det(X^T X) = 5 \neq 0$, the matrix is invertible, and we can compute the least squares solution

Multicollinearity Case:

$$X^T X = \begin{bmatrix} 4 & 6 \\ 6 & 9 \end{bmatrix} \tag{484}$$

$$\det(X^T X) = 4(9) - 6(6) = 36 - 36 = 0 \tag{485}$$

Features are perfectly collinear! Cannot compute inverse. Need regularization.

2. Multivariate Gaussian Distribution:

Probability density involves determinant of covariance:

$$p(x) = \frac{1}{(2\pi)^{d/2}|\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(x-\mu)^T \Sigma^{-1}(x-\mu)\right) \tag{486}$$

Example:

Covariance matrix:

$$\Sigma = \begin{bmatrix} 4 & 2 \\ 2 & 3 \end{bmatrix} \tag{487}$$

$$|\Sigma| = 4(3) - 2(2) = 12 - 4 = 8 \tag{488}$$

$$|\Sigma|^{1/2} = \sqrt{8} \approx 2.828 \tag{489}$$

This normalization constant ensures probability integrates to 1.

3. Volume Transformation in Normalizing Flows:

Change of variables formula:

$$p_Y(y) = p_X(x) \left| \det\left(\frac{\partial f}{\partial x}\right) \right|^{-1} \tag{490}$$

Example - Simple 2D Transformation:

Transformation: $y_1 = 2x_1$, $y_2 = 3x_2$

Jacobian:

$$J = \begin{bmatrix} \frac{\partial y_1}{\partial x_1} & \frac{\partial y_1}{\partial x_2} \\ \frac{\partial y_2}{\partial x_1} & \frac{\partial y_2}{\partial x_2} \end{bmatrix} = \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix} \tag{491}$$

$$|\det(J)| = |2 \times 3| = 6 \tag{492}$$

Volume is scaled by factor of 6 under this transformation.

4. Principal Component Analysis - Variance Explanation:

Total variance is related to determinant of covariance:

$$\text{Generalized variance} = \det(\Sigma) \tag{493}$$

Example:

Original covariance:

$$\Sigma = \begin{bmatrix} 9 & 4 \\ 4 & 4 \end{bmatrix} \tag{494}$$

$$\det(\Sigma) = 9(4) - 4(4) = 36 - 16 = 20 \tag{495}$$

After removing redundancy (PCA):

$$\Sigma_{reduced} = \begin{bmatrix} 10 & 0 \\ 0 & 2 \end{bmatrix} \tag{496}$$

$$\det(\Sigma_{reduced}) = 10 \times 2 = 20 \tag{497}$$

Same generalized variance, but with uncorrelated components

5. Checking Stability of Neural Network Weight Matrix:

For RNN stability, eigenvalues of weight matrix should have magnitude ≤ 1.

Quick check using determinant:

If $|\det(W)| > 1$, at least one eigenvalue has magnitude > 1 (potential exploding gradients).

If $|\det(W)| < 1$, productof eigenvalue magnitudes is < 1 (potential vanishing gradients if all eigenvalues are small).

Example:

$$W = \begin{bmatrix} 0.5 & 0.2 \\ 0.3 & 0.6 \end{bmatrix} \tag{498}$$

$$\det(W) = 0.5(0.6) - 0.2(0.3) = 0.3 - 0.06 = 0.24 \tag{499}$$

$|\det(W)| = 0.24 < 1$ suggests gradients might vanish. Consider orthogonal initialization.

13 Summary

This comprehensive guide covered matrix operations in machine learning and deep learning:

- **Scalar Operations:** Uniform scaling/shifting for learning rates, normalization, weight decay, and temperature scaling
- **Matrix Addition/Subtraction:** Ensemble methods, residual connections, error computation, and batch normalization
- **Matrix Multiplication:** Linear transformations, forward propagation, batch processing, and attention mechanisms
- **Transpose:** Dot products, normal equations, backpropagation, covariance, and self-attention
- **Determinant:** Invertibility checking, multivariate distributions, volume transformations, and stability analysis

All examples included complete step-by-step calculations with practical ML/DL applications.